July 2026

World-Time Compute with Verified Code World Models

James Schwoebel[1,*] • Ingrida Semenec, PhD[1] • Jenia Rousseva, PhD[1] • Marcos Ortiz, PhD[1] • Collin Overbay[1] • Christopher Klaus, PhD[7] • Anderson Edmond[8] • Manish Bhatt[4,6] • Rome Thorstenson[3] • Jessica Tsai, MD, PhD[1] • Martin G. Frasch, MD, PhD[2,5]

[1] Quome, Inc. [2] University of Washington [3] Rafter Labs, Inc. [4] OWASP [5] Health Stream Analytics, LLC [6] IEEE Computer Society [7] Fusen World LLC [8] Independent [*] Corresponding author

ABSTRACT

A large language model learns to generalize across a domain only after it sees many real, labeled examples from that domain, and in most domains those examples are scarce. We study a cheap way to make them. When a domain's dynamics can be written as code, one template turns into many *world models* (simulators whose dynamics are executable, verifiable programs over symbolic state), and each one is an endless source of *exactly*-labeled trajectories. We fine-tune an LLM on trajectories run through many such world models of a domain (**world-time compute**, the training-time cousin of test-time compute), and this raises how well it generalizes to *held-out* worlds it never trained on (shown on synthesized world families). The gains are largest where the model is weakest (+29 points at 0.5B; the largest model's lift is within noise, which fits task saturation). You can trust the labels because the worlds are *verified code*, not learned approximations: dynamics that are synthesized and then checked stay exact over 20-step rollouts and answer 10× out-of-distribution probes exactly (100%), while per-step LLM prediction and a trained MLP pile up error and collapse. Unlike domain randomization (which varies one simulator's parameters), we author and verify each world on its own; a corrupted-label control shows that label *exactness*, not task variety, drives the held-out gains.

On real, human-authored benchmarks (ARC-AGI grids, List Functions, CLRS algorithms) the same lever works as *per-world* test-time training: it grows with compute and collapses under label corruption. On one coherent real family (List Functions) the harder *cross-world* form works too: a single adapter trained on 128 *disjoint* worlds generalizes to held-out worlds it never saw, reaching 40% against 6% for a corrupted-label control (a +34-point exactness-gated lift, CI [29, 39]). The gain follows a *descriptive* regularity—a curve that saturates, not a law: it is largest for *few-step* reasoning and small, weak models, and it fades for long reasoning chains, for tasks read off rich perception, and as a task saturates; transfer across tasks is weak when worlds share no skill. The worlds are authored and served by **OpenWorld**, a zero-dependency framework that makes it cheap to assemble the *many* worlds a domain needs (the framework is a companion paper). **Scope: the symbolic-state regime**: where you can write down the dynamics, verified code makes a *correct* world model a zero-training prototype; pixel-native domains stay the territory of learned models. All code, recipes, and this manuscript regenerate from one public repository.

Keywords: world models; code world models; neuro-symbolic AI; program synthesis; value alignment; agents-as-a-judge; local LLMs.

Code & data: `github.com/quome-cloud/openworld` **Correspondence:** jim@quome.com

1 Introduction

Manufacturing experience. A language model learns to generalize across a domain only after it sees many real, labeled examples of it—and in most domains those examples are scarce and costly. We study a cheap way to make them. When a domain's dynamics can be written as code, a single template turns into many *world models*—simulators whose dynamics are executable, verifiable programs over symbolic state—and each one generates exactly-labeled trajectories without end. Fine-tuning a model on trajectories run through many such worlds of a domain—*world-time compute* (§4.3), the training-time analogue of test-time compute—raises how well it generalizes to held-out worlds it never trained on. Two things have to hold. First, the worlds must not lie: a world teaches the right rules only if its dynamics are correct, so we build them as verified code rather than learned approximations. A model writes the transition as a program, and we accept it only after it parses with the required signature, runs in a sandbox without breaking the world's declared invariants, and (optionally) survives a second model's critique against the rules. The accepted program stays exact over long rollouts with no compounding error (Figure 1A). Second, world-time compute needs many worlds, and that is practical only if each one is cheap to build. The tool that makes it cheap is OpenWorld, a zero-dependency framework that turns authoring a verified world model (perceive → world → emit → act, served and shareable) into a minutes-scale, push-button task (the companion framework paper [38]). No general tool has made authoring the many verified world models this needs cheap; OpenWorld is that tool, and we use it here to manufacture training experience.

A world model is an agent's internal simulator: given a state and an action, it predicts the next state (and/or observation), which lets the agent plan "in imagination" instead of by trial in the real world [18]. The whole field shares that *genus*; where research programs split is *how the model represents state and how it is obtained*, and today the term names two largely separate *species*. The dominant, most visible one is *perceptual and generative*: a neural network learned from large amounts of data, whose state is a continuous latent (or pixels, video, or a 3D scene) and whose output is a *generated observation*. It runs from the V–M–C triad of Ha and Schmidhuber [18] through the RSSM-based Dreamer family [19], value-equivalent planning in MuZero [37], autoregressive token models such as IRIS [27] and Δ-IRIS [28], diffusion dynamics in DIAMOND [3], and joint-embedding predictive video models that forecast in representation space such as V-JEPA [5]. Most visibly in recent generative AI, the species now reaches interactive generative environments and "spatial intelligence"—internet-scale video and 3D world generators such as Genie [6], video-generation-as-simulator (Sora) [31], and Fei-Fei Li's World Labs [15]. Powerful as these are, they carry three built-in costs: they are *data-hungry* (millions of environment steps, or internet-scale video), *compounding* (every learned transition adds error that builds up over a rollout), and *opaque* (both dynamics and values live in continuous weights you cannot inspect, edit, or audit).

What "world model" means in this paper. The term has come to suggest generative, pixel- and 3D-native systems, but we use a simpler, narrower sense—a forward predictor of the next *state*, for planning—and we hold the state to a *symbolic* object: a typed record of named variables (counts, prices, positions, flags), not pixels, latents, or 3D geometry. Our world models are therefore the second species, *Code World Models* (CWMs): the transition is an explicit, human-readable *program* over that symbolic state, synthesized by an LLM from a declared rule set and verified before use, and a rollout returns the next symbolic state exactly rather than a generated image, with zero training data. This is the same genus as the perceptual models above (a simulator the agent rolls forward) but a different species—and one we mean to be complementary. OpenWorld aims at domains you can *specify* symbolically (inventories, markets, schedules, negotiations, physics with

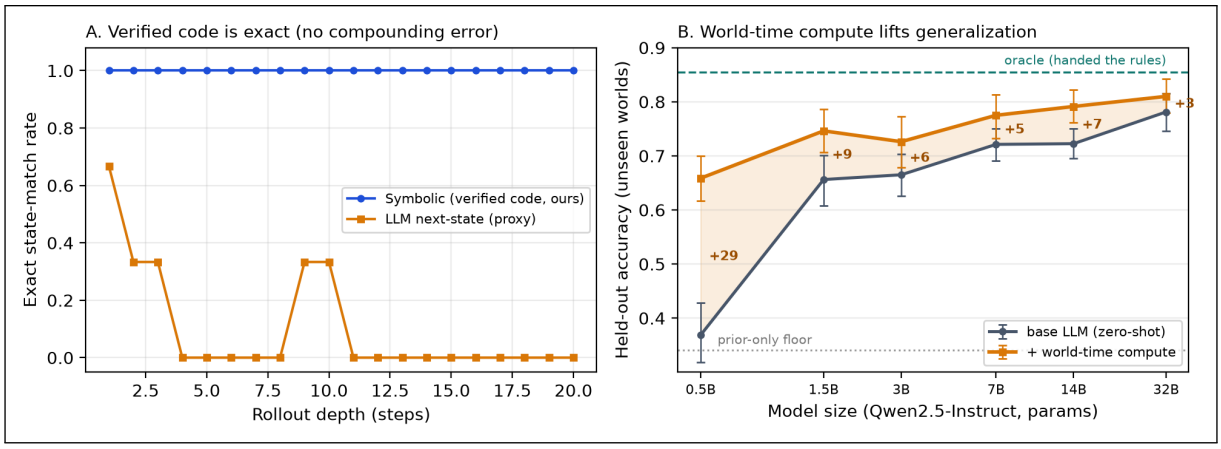

Figure 1: *A:* The task is to predict a world's next symbolic state from the current state and an action (here the sprint backlog/debt/bug world, wired to a ground-truth oracle). Dynamics synthesized and verified once as code (blue) match the oracle exactly at every depth of a 20-step rollout, while per-step LLM next-state prediction (orange)—a structural stand-in for learned neural dynamics—first goes wrong at step 2.3 on average and never finishes an exact rollout. (Panel A is the per-depth exact-match rate over n=3 rollouts, so the orange trace is noisy and non-monotonic; the solid claim is the contrast with code's exact-at-every-depth line, not the shape of the orange.) Programs encode rules, not statistics, so the compounding-error failure of learned world models is ruled out by construction, at zero training cost. *B (payoff):* what that exactness buys—world-time compute. Fine-tuning an LLM on trajectories run through 60 verified train world models raises held-out diagnostic accuracy on 20 unseen worlds toward the rules-given oracle, with the largest gain for the smallest models (+29 points at 0.5B, shrinking to +3 at 32B)—a generalization lever that is strongest where capability is scarcest (bars: 95% bootstrap CIs over held-out specialties; dotted line: the prior-only floor; E74, §4.3).

known equations); it is not a pixel-native, perceptual, or open-world-3D model, and it does not generate images, video, or scenes. Where the spatial-intelligence systems *learn what cannot be told*, OpenWorld *writes down what can be told, and verifies it.*

World, action, agent, simulation. The world model is only one of four separate objects. A *world* is the environment-as-simulator: a symbolic state, a discrete set of *actions*, and the verified transition that maps a (state, action) pair to the next state (plus optional perception/emit boundaries). An *agent* is a *separate* policy (an LLM planner or a fixed rule) that picks actions toward a goal; it is a *user* of a world, not a part of it, so one world serves any agent and you can swap the planner without re-synthesizing dynamics. A *simulation* closes the loop: it puts agents in a world and repeatedly observes, picks an action, and steps the world, producing a trajectory scored by objectives. The textbook definition puts the world model *inside* the agent (the simulator it plans with) and sets it apart from the true environment; the gap between the two (model bias, sim-to-real error) is exactly what limits learned world models. A verified code world model does not so much remove that gap as *relocate* it: once the dynamics are declared symbolically and the accepted program reproduces them exactly, the model an agent plans through and the (symbolic) environment it acts in can be the *same* verified transition, so the dynamics add *no* rollout error—any error that remains lives entirely in the *specification* and in *perception* (the symbol-grounding boundary; see the companion framework paper [38]), not in the transition. This is why the same `World` serves, unchanged, both as the ground-truth environment and as the model an agent plans through in our planning experiments; it is also why this paper is scoped to *specifiable* domains.

This Code World Model bargain (programs instead of weights) buys verifiability, compute-cheap

rollouts, and out-of-distribution generalization by construction [44, 13, 33, 22]. Yet CWM research has stayed a set of one-off systems aimed at particular benchmarks—built for evaluation or post-training, rather than giving agents a reusable framework to use them during inference. No general, lightweight framework has let a practitioner declare a world, have a local model write and verify its dynamics, steer behavior through declared value weights, and optimize the whole setup against a goal—the workflow that made supervised learning ubiquitous.

OpenWorld is that framework, and this paper benchmarks the paradigm it puts into practice against the learned-dynamics alternative. The framework adds four mechanisms: (i) a *plan–generate–verify relay* in which a generator LLM writes transition code that must pass syntactic checks, sandboxed smoke-runs, and declared invariants, and optionally a second-model critic, before we accept it; (ii) *open value specification*—objectives are scoring functions weighted by tunable dials, so the moral configuration is an artifact you can edit rather than a frozen weight, which speaks directly to the specification trap [42]; (iii) an *automated tuner* that searches world design, policy knobs, and moral dials together (random, local-refinement, and Optuna TPE [1] strategies); and (iv) *agents-as-a-judge* [52, 51]: an LLM judge that picks among candidate behaviors and grades trajectories against written rubrics.

A survey of the world-model landscape shows distinct strategy families with a recurring tension—fidelity and scale on one side; verifiability, sample efficiency, and steerability on the other (a positioning table against the representative strategies appears in the companion framework paper).

Explicit vs. implicit world models, and world models as evaluators. Two strands of recent work sharpen what a verified, explicit world model buys. The first asks whether a trained model even holds a coherent world model: implicit world models read out of a generative model's predictions are often incoherent [46], and whether a network represents one inside at all is contested [23]—a question OpenWorld steps around, because the model is the human-readable transition program, so coherence is a property you check, not one you infer. Video-generation world models likewise miss the governing physical law even at scale [20]; OpenWorld instead declares and verifies the law, trading learned breadth for exactness where the law is known. The second strand uses world models as policy-evaluation environments [34]; because those environments are learned, a policy's score picks up the simulator's error, whereas an OpenWorld evaluation world runs exactly, so model error does not confound the score.

Contributions. Three, all around the world-time-compute thesis:

1. **World-time compute on real domains, with cross-world transfer.** A verified world model is *training signal*: fine-tuning an LLM on trajectories run through many world models of a domain raises its generalization to held-out worlds it never trained on. The new cross-world axis—train on a family, generalize to worlds you have not seen—holds on a real, human-authored domain (List Functions, E84): one adapter trained on 128 disjoint worlds reaches 40% on 60 held-out worlds versus 6% for a corrupted-label control—a +34-point exactness-gated lift (CI [29, 39], 3 seeds). On synthesized families the gain is largest where capability is scarcest (+29 at 0.5B, within noise by 32B as the task saturates). A corrupted-label control shows that exact labels, not task variety alone, are the lever—which is what sets this apart from domain randomization.

2. **Why it must be verified code.** Running a world is safe only if the worlds do not lie. On three ground-truth worlds, verified synthesized dynamics are exact on 24/24 20-step rollouts (95% CI [0.86, 1.00]) and answer 10× out-of-distribution probes at 100% from zero transitions,

while the LLM next-state proxy completes 0/24 and a 10,000-transition MLP scores 0% out of distribution. We *measure* verification, we do not assume it (each gate buys +0.24 probe accuracy over blind acceptance), and it certifies executable, invariant-respecting code, not rule-correctness (on a weak 3B generator the gate accepts every program yet none is branch-exact). Handed the dynamics, planning through the verified model beats planning through a hallucinating one and solves 0-shot what trained DreamerV3 / V-JEPA-2 need 10^4–10^5 steps to learn [19] (a scope property, not a contest; §4.13).

3. **The substrate, and full reproducibility.** OpenWorld, a zero-dependency framework, turns authoring and serving the many verified worlds world-time compute needs into a minutes-scale activity; the framework, its 100-world prototyping benchmark, and the broader world zoo are the companion paper [38]. All 79 experiments are seed-fixed, use paired tests, and regenerate from one public repository.

Positioning in the field. OpenWorld builds on the *programmatic* code-world-model program [44, 13, 33, 22]—LLM-synthesized, verified code dynamics, as distinct from *neural* code world models that learn dynamics in weights [12] and from LLM synthesis of symbolic world models via test-time scaling [48]—but its contribution is what that machinery makes possible: world-time compute, which turns many verified worlds into exactly-labeled training signal. Set against the *learned* world-model families—latent-dynamics models (Dreamer [19], MuZero [37]), tokenized and diffusion video models (IRIS [27], DIAMOND [3], Genie [6], Sora [31]), and joint-embedding predictors (V-JEPA [5])—it trades perceptual breadth for an auditable, exactly-executable symbolic transition with no compounding rollout error. Its closest relatives are the *self-training* methods that retrain a model on its own verifier-filtered outputs (STaR [50], ReST [16, 40], expert iteration [4], rejection-sampling fine-tuning [49], verifiable rewards [21], and test-time reinforcement learning on unlabeled data via majority-vote pseudo-rewards [53]); we differ in what does the verifying (a full world model with dynamics, so the unit is a verified trajectory, not a checked answer) and in the *generalization axis* (held-out worlds in a family). It also differs from domain randomization and procedural task generation [45, 11, 29]—and from foundation-model-generated code environments for open-ended learning [14]—where the experience comes from a single (often hand-coded or learned) simulator, or from generated environments picked for how interesting they are rather than checked for label-exactness. Here every world is synthesized and verified on its own, and the open question we pose is whether that label exactness, not task variety, drives the held-out gains. The labels are exact only *relative to the synthesized world model*—"verified" means the transition respects the declared invariants and passes the probe set, not that it provably equals the domain's true dynamics (which we may only observe in part). So a world whose declared rules are subtly wrong is a confidently-wrong teacher; the corrupted-label ablation (§4.4) measures exactly that failure mode. The supporting machinery extends the open-specification stance of the value-alignment literature [42, 35] and judge-based behavior selection [52]. We benchmark on consumer hardware with 7B/3B local models, the regime where learned world models are least accessible and training-free approaches matter most.

2 Problem Setting

We aim at symbolic-state environments. In these worlds the state is a JSON-serializable structure, the action set is discrete, and the dynamics follow rules you can declare. This covers operational simulations (triage, negotiation, portfolios, sprints) and program repair, but it does not cover pixel-native domains; we come back to this boundary in the limitations. The practitioner declares

a world—its state schema, its actions, and its rules in plain language. The framework must then build a simulator that is faithful, fast, and auditable, plus the tools to steer and optimize behavior inside it. What can go wrong here is *model error itself*: made-up transitions, silently wrong code, badly specified rewards, and value drift.

A world model, formally. An OpenWorld world model is a tuple

$$W \;=\; \big(\mathcal{S},\ \mathcal{A},\ s_0,\ \tau,\ \{(g_i, w_i)\}_i,\ \mathcal{I}\big),$$

with symbolic states $\mathcal{S}$ (JSON-serializable maps), a discrete action set $\mathcal{A}$, an initial state s_0, a *deterministic* transition program $\tau : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ (illegal actions do nothing, so τ is total; stochastic worlds thread a seed through, so rollouts stay replayable), weighted objectives $\sum_i w_i o_i$ that score transitions, and invariants $\mathcal{I}$ that must hold on every reachable state. The practitioner declares everything but τ—the schema of s_0, the actions, the natural-language rules R, the objectives with their *dials*, and the invariants—and the framework must *produce* τ. The framework's full `World` anatomy is wider—*perception* → state → actions → transition → reward (the companion framework paper [38]); here we work with its dynamics-and-scoring core and treat perception, the map from a raw observation to a symbolic state, as a separate concern. Making perception a first-class part—grounding a world directly from raw observation such as pixels—is the focus of a companion ARC-AGI-3 paper [39] (under review).

World-time compute does not roll the transition forward. Instead the agent takes a world model *it built and verified* as ground truth, and it trains on the exactly-labeled examples that world hands it: an input o paired with the answer y the world fixes—the disease a generative world drew (E74), the output a real task's hidden rule determines (List Functions, ARC), or the next state a verified program computes (program repair). The label is exact because the agent reads it off its own verified world rather than guessing it—exact *relative to* that world. The question this paper sets out to measure is how far an agent that builds its own worlds and learns from them can then handle wide-ranging problems.

OpenWorld produces τ by *synthesis under verification*, not by learning. An LLM proposes a program $\hat{\tau}$ from R. We accept it only if it parses with the required signature, runs in a sandbox that keeps every invariant on a probe set, and, optionally, gets past a second model's critique. This certifies that $\hat{\tau}$ is invariant-consistent, not that it equals the true dynamics $\tau^\star$. But once we accept it, $\hat{\tau}$ is fixed code: wherever it matches $\tau^\star$ on the reachable set, a t-step rollout is exact at every depth, bit-for-bit. A transition predicted step by step with single-step error rate ϵ instead stays exact for only about $(1-\epsilon)^t$ of a t-step rollout, decaying with depth—the compounding error that a fixed program avoids by construction.

3 Experimental Setup

Worlds with ground truth. Three deterministic oracle worlds drive every dynamics comparison: *sprint* (backlog/debt/bug dynamics with an integer-division interaction), *orchard* (a resource pool with per-agent tallies), and *triage* (queues, a clock, and deterioration events). The oracles are hand-written `FunctionTransitions`. We compare synthesized and learned-style engines against them exactly, including on a fixed probe suite that exercises the clamping, empty-queue, and interaction branches.

The coding world. Program repair is the flagship use case: it is well-known, objectively scored, and symbolic by nature [7]. Each of 20 tasks is a buggy Python function plus a hidden test

suite that spans classic defect archetypes (off-by-one ranges and binary-search bounds, inverted comparisons, missing normalization, wrong base cases, unsorted medians, order-destroying dedup, early imbalance, whitespace splitting, accumulator resets, integer-division truncation, overlapping counts, dropped final runs, and degenerate-input handling). The world's transition is test execution: a submitted patch runs bit-exactly in a sandbox with a wall-clock alarm (a looping patch fails instead of hanging), and failing-test feedback enters the state for the next attempt.

Benchmark-scale repair (E28–E29). Beyond the single-function benchmark, two SWE-bench-style suites test repair at module scale. Each instance carries a natural-language issue report (symptoms and a repro, never the fix), a buggy module (functions or a stateful class), two hidden suites (`fail_to_pass` exercising the defect, `pass_to_pass` guarding against regressions), and an explicit world specification whose `submit_patch` dynamics run both suites bit-exactly. Solving requires zero failures in both, so a fix that breaks a passing test does not count. The *atomic* suite (E28, $n = 6$) plants one issue-described defect per instance; the *staged* suite (E29, $n = 15$) plants a second, hidden defect that shows up as a new failing test only once the issue-visible defect is fixed. Both run a paired ablation across the `qwen2.5` ladder (1.5B/3B/7B): the same model *single-shot* (one completion from issue + module) versus *in-world* (up to 4 `submit_patch` steps; the first prompt is identical to single-shot, and exact failing-test feedback enters from the second attempt onward). An expanded 20-instance atomic suite with extra regression traps ships in the repository (`datasets/openworld-repairbench/`).

Composition, nesting, and changing rules (E30–E32). Three experiments exercise `CompositeWorld`, a wrapper whose state nests its children's states under namespace keys. Children run *unmodified*: the composite slices out a child's namespace, steps the child's own transition, and writes it back. Coupling is explicit. It runs sideways through `Bridge`s (an ordinary transition over the two-slot dict of the coupled children's states, so the same synthesis-and-verification pipeline applies to bridges unchanged), and upward through `Aggregator`s (derived from the leaves, never simulated); agents travel between children over `Route`s whose crossing effects (tolls, vetoes) are themselves verifiable transitions. `PhasedTransition` handles rules that change over time: ordered (trigger, transition) phases that advance in sequence and cannot reverse, with every phase built and verified before the run. E30 holds one system fixed—16 internal rules plus four ring couplings—and varies only how we pose the synthesis: one monolithic prompt over a flat namespaced state versus eight small prompts (four 4-rule sectors, four one-rule bridges), each verified on its own and then assembled into a `CompositeWorld`. Both conditions score on the same ground-truth probe suite (7B generator, three replicates each). E31 replays a 20-step script (leaf actions, ticks, and two travel attempts across a toll route) on a three-level composite against an independent flat-dict oracle that imports nothing from the composition machinery, checking leaf exactness, aggregator consistency, money conservation, and agent-registry agreement at every step. E32 declares a market whose policy changes at a fixed step mid-rollout, and it compares phased synthesis (each phase from its own rule text alone), monolithic synthesis from the combined rules-with-change text, and the LLM next-state proxy, on closed-loop rollouts that cross the boundary.

Models and hardware. All experiments use local models served by Ollama on an Apple-silicon laptop: `qwen2.5:7b` (generator, judge, critic in E3), `qwen2.5:3b` (small generator in E2/E3), `qwen2.5:1.5b` (repair agent in E5/E6/E13/E17), and—for cross-family replication (E16)—`llama3.1:8b` (Meta) and `gemma2:9b` (Google). The trained baselines in E12/E19 are plain numpy models. We use no cloud APIs, no fine-tuning, and no training compute beyond the seconds-scale fitting of the

baselines.

Baselines. We represent the learned-dynamics family in two ways. (i) `LLMTransition`: the same backbone predicting each next state directly from the same description and rules—it shares the symbolic engine's knowledge but has the per-step stochastic prediction structure of learned models. (ii) Trained models (E12): a two-hidden-layer MLP and a 1-nearest-neighbor memorizer, each trained on $K \in \{100, 1000, 10000\}$ environment transitions collected by a random policy—real learned dynamics with a measured sample-efficiency curve. We say plainly that neither is a trained Dreamer; reimplementing the pixel-native family at laptop scale is infeasible, and that is itself part of the comparison (DreamerV3-class systems require millions of environment steps [19]).

Metrics and statistics. Rate metrics carry 95% Wilson confidence intervals. Dynamics fidelity uses exact-state match (every field equal), the first-divergence step, and final-state L^1 error. Program repair reports pass@1 and pass@budget (4 attempts); the controlled selection comparison (E13) is a paired design on shared candidate sets with an exact two-sided McNemar test. Judge alignment uses Spearman rank correlation with permutation p-values (10,000 shuffles). We fix the seeds in each script; every number in this paper regenerates via `python3 scripts/make_paper_assets.py`.

Calibration, pilots, and internal review. We report the design changes we made after pilots rather than hide them: (a) the verifier ablation (E3) uses the 3B generator at high temperature because the 7B generator's first attempts passed every gate, leaving nothing for the conditions to tell apart; (b) the repair agent in E5/E6/E13 is the 1.5B model because both the 7B and 3B agents saturated the original benchmark (pass@1 = 100%)—with failing-test feedback in the state, these archetypes are easy for capable models. The capability ladder (1.5B: 70%, 3B and 7B: 100% on the original suite) is itself a finding. All LLM-dependent numbers are specific to the stated Ollama snapshots and Q4_K_M quantization.

4 Results

4.1 Head-to-head: symbolic vs. learned-style dynamics

This section measures *dynamics fidelity*—how well we predict a world's next state from the current state and an action—not the program-repair task of the coding world. The practitioner writes down each world's rules. The question is how to turn those written rules into a faithful simulator. The symbolic engine compiles the rules to a verified `transition` program once, then runs it. The learned-style engine predicts each next state directly. The sprint world's `bugs += debt//4` coupling is an arbitrary *declared* rule, not a natural law to be guessed. The point is not that a model should see it coming. The point is that once you declare it, compiling it to code runs it exactly, while predicting it step by step piles up error.

Table 1 and Figure 1 show the main comparison (E1, E4, E10), and Table 2 runs the same protocol across all three instrumented worlds, with eight action scripts each (E11). The verified synthesized engines are exact on 24/24 rollouts (95% CI [0.86, 1.00]), against 0/24 (CI [0.00, 0.14]) for per-step LLM prediction. And because planning runs Python instead of a forward pass, they roll out at 14,997 steps/s against the LLM engine's 0.32 steps/s, after a one-time synthesis cost of a few seconds.

Engine	First divergence (step)	Final L1 error	OOD exact (10×)	Steps/s
Symbolic (verified code, ours)	21.0	0.0	100%	14,997
LLM next-state (proxy)	2.3	10.3	40%	0

Table 1: Head-to-head engine comparison on the sprint world (E1, E4, E10): mean first-divergence step and final L^1 error over three 20-step rollouts against a ground-truth oracle; exact transition accuracy on ten 10× out-of-distribution probes; and rollout throughput. A first divergence of 21 means no divergence occurred within the rollout.

World	Code: exact rollouts	LLM: exact rollouts	LLM first divergence
orchard	8/8	0/8	11.2
sprint	8/8	0/8	1.4
triage	8/8	0/8	2.0
Total	**24/24** [0.86, 1.00]	0/24 [0.00, 0.14]	—

Table 2: Multi-world replication (E11): exact 20-step rollouts per world, eight scripts each, dynamics synthesized once per world by the 7B generator. Totals carry 95% Wilson CIs.

4.2 The central result

Verified code synthesis eliminates compounding rollout error at zero training cost. The reason is structural, not statistical. A learned (or per-step predicted) transition is sampled fresh each step, so with per-step error ϵ the probability that a t-step rollout has *diverged* grows as $1-(1-\epsilon)^t$ (one minus the exactness probability $(1-\epsilon)^t$): with the measured single-transition error of the LLM engine (60% of probes wrong), divergence by step 2.3 is just what the arithmetic predicts, and twenty-step plans are fiction (final L^1 error 10.3). The symbolic engine pays its whole error budget once, at synthesis time, where you can audit it: if the accepted program is right, every rollout of any depth is right, bit-exactly. This is the CWM hypothesis [44, 22] made measurable on consumer hardware.

The single-number divergence 2.3 comes from one world (sprint, n=3 rollouts, one seed), and it is not representative: across the three worlds (E11, 24 rollouts) the LLM proxy's first divergence ranges from step ≈1.4 (sprint) and 2.0 (triage) to ≈11 (orchard), because compounding depends on the per-step error rate, and that rate varies by world. The claim that holds regardless of world is the exact-match contrast, not the mean divergence step: code engines finish 24/24 of 24 twenty-step rollouts bit-exactly (95% CI [0.86, 1.00]) versus 0/24 for the LLM proxy. We treat the divergence figure as an illustration of the mechanism and the exact-match rate as the result.

4.3 World-time compute: traversing world models to generalize from fewer examples

The results so far buy a *correct* world model cheaply. A verified world model is also a *generator*: it makes unlimited, exactly-labeled experience and shows its own rules, so an agent can *traverse* many worlds of a domain and distill the shared skill. We call spending compute this way—fine-tuning a model on trajectories drawn from traversed, verified world models rather than on scarce real examples—**world-time compute**, a training-time analogue of test-time compute [41, 30, 17] (and, when spent per world at inference, of test-time training (TTT) [2]).

We test it on a *family* of diagnosis worlds (E74): one partially observable Markov decision process (POMDP) template (a hidden disease; order tests to reveal symptoms; commit a diag-

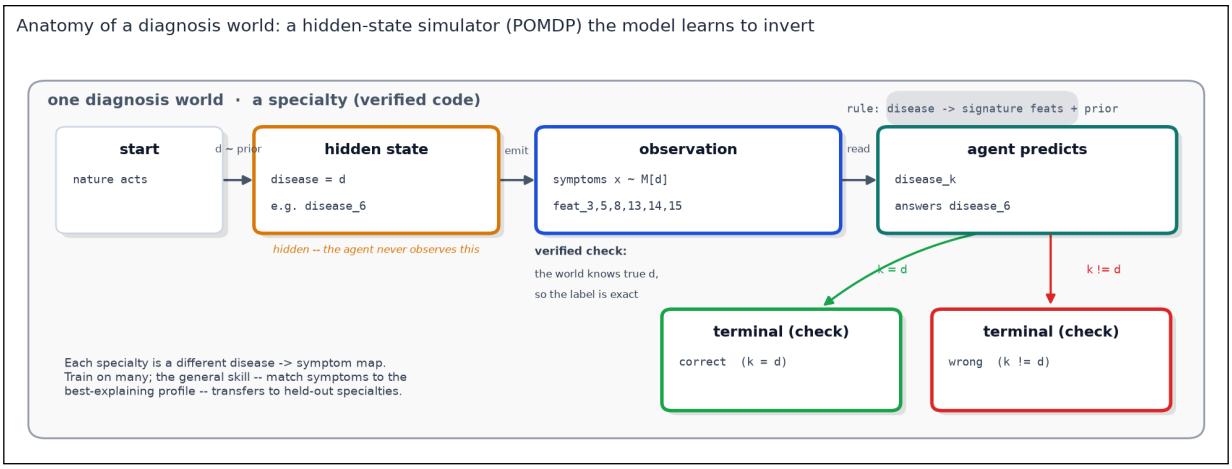

Figure 2: **Anatomy of one diagnosis world (E74–E79).** Each specialty is a small hidden-state simulator: nature draws a hidden disease from a prior, the world emits symptoms from that disease's signature features (plus background noise), and the agent must invert the observation to name the disease. The agent treats this world—which it built and verified—as ground truth, so the disease the world drew is a known, exact label (relative to that world): a traversed world distils a trusted rule, not a hallucinated one. Each specialty is a different symptom → disease map; training on many and testing on held-out specialties forces the general skill (match symptoms to the best-explaining profile) rather than memorisation.

nosis) instantiated as 60+20 specialties that share a goal and action grammar but differ in their symptom → disease structure. We hold out whole specialties—a world-level train/test split, as in supervised learning: fine-tune a model on traversed train specialties, then measure diagnostic accuracy on 20 unseen specialties (800 cases). Two brackets frame the result: a prior-only floor (34%) and an oracle handed the rules (86%). Figure 2 walks through one such world—a hidden-state POMDP the model learns to invert. To be exact about what transfers: each test prompt states that specialty's profiles (its rules) in context, and the base baseline sees the same prompt—so the lift is not the fine-tuned model reading a table the baseline cannot (both see the rules). The lift is fine-tuning teaching the model to apply the given rules in a way that carries to symptom → disease maps it never trained on. This is the in-context regime of test-time training, one rung more general (across rule-sets, not within one); internalizing fully *latent* dynamics is harder and we do not claim it here.

Two findings (Figure 1B). *(i) Generalization, not memorization:* world-time compute lifts held-out accuracy at every model size—from 37% to 66% at 0.5B, and from 78% to 81% at 32B—moving the model toward the rules-given oracle on specialties it never trained on. *(ii) It is a small-model multiplier:* the gain is largest where capability is scarcest (+29 points at 0.5B, bootstrap CI over held-out specialties well clear of zero) and *shrinks* as larger models approach the rules-given ceiling—by 32B the lift is only +3 points and its CI includes zero, because the task is nearly saturated (oracle 86%). Does the gain come back once capable models are given headroom—a deliberately harder world family? That is a clean falsifiable follow-up we are running. An offline analogue confirms the distilled structure is a sample-efficiency win either way: a learner that has seen the family diagnoses a held-out specialty from fewer cases than one learning from scratch (70% vs 68% at a single case per disease).

The family is what makes this work. Fine-tuning on *heterogeneous*, unrelated worlds that share no task structure yields no such transfer (E71, E73): there is no shared skill to distill, so a policy expert on one world does no better than chance on another. World-time compute pays off exactly when the traversed worlds form a domain—then more traversal becomes a lever on

generalization, complementary to train-time data and test-time search. (Scope: the model-size sweep mixes precisions—0.5B–7B are bf16 LoRA, the larger sizes 4-bit QLoRA—and the per-number-of-worlds axis saturates quickly, so we claim a compute *lever*, not a clean world-count scaling law.)

4.4 Scaling the lever, and a coding-world transfer test

Two follow-ups sharpen the principle. Does it scale with the number of worlds? And does it hold on a different use case, one authored as real verified-code worlds?

World-count scaling (E76). We fix the model (7B) and the hard family and vary only the number of train specialties traversed, holding the test specialties fixed (Figure 4). Too few worlds *hurt*: the fine-tune overfits a narrow slice, below the 48% base. After an initial dip the gain rises monotonically and is *still climbing at 512 worlds* (59%, +10 points), heading toward the 69% oracle—no plateau. So world-time compute is a genuine scaling lever in the number of worlds, when the task has headroom. The offline empirical-Bayes prior saturated at ≈2 specialties precisely because it has none of the world's *skill* left to keep learning.

Exact labels are the lever (E78b). Scaling shows that more worlds help; this ablation shows why the worlds must be verified. We hold the world count fixed (256 train specialties) and keep the patients, prompts, and example count identical, and vary only the training labels: a fraction p of each world's diagnoses is replaced by a confusable disease drawn from the posterior over the others—the structured error a non-verified or learned world model makes when its rules are slightly wrong. The held-out test labels stay exact. Held-out accuracy falls monotonically as corruption rises, from 35% at p=0 (verified) down to 19% at p=1—a fully-wrong world gives *no* generalization gain—in fact at or below the prior-only floor (23%)—with a smooth penalty in between (Figure 3). So what drives the effect is label *exactness*, not task variety—the property that sets world-time compute apart from domain randomization, and the reason the worlds must be verified code (3 seeds per level; per-seed values shown in Figure 3).

A coding world family (E77). Coding is the cleanest verified-code world we have: the transition is execution and the oracle is the tests passing. We had a model author 164 function-implementation tasks, each admitted only after its reference solution passes its own sandboxed `pytest` oracle, and we fine-tuned on the traversed worlds (Table 3). *In-domain*, world-time compute lifts best-of-k at every model size (7B pass@5 84%→95%). On HumanEval—a real, out-of-distribution benchmark—it hurts greedy pass@1 (7B 78%→70%; the synthetic fine-tune narrows the single best guess) yet *transfers positively at pass@5* (87%→91%) from only 164 worlds. That fits E76: 164 worlds is below the count where transfer becomes strong, so the path to real-benchmark gains is to scale the world family, not to abandon the approach.

4.5 Real data: world-time compute on ARC-AGI (E80)

So far, every world-time-compute result runs on worlds we wrote ourselves (diagnosis, coding). That invites the sharpest objection: maybe the gains come from our generator, not from the world models. So we move the mechanism to a benchmark we did not write—**ARC-AGI** [8, 9], the human-authored abstraction-and-reasoning corpus (400 training + 100 evaluation tasks; we use ARC-AGI-1, with ARC-AGI-2 [10] a harder successor). It fits unusually well. Each task is a world—a hidden grid-transformation rule shown only through a few input → output demonstrations. So

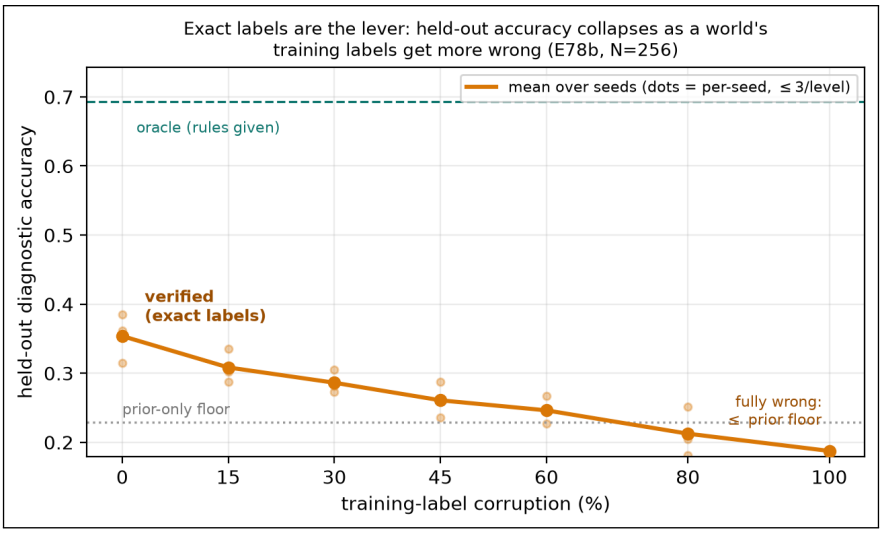

Figure 3: **Exact labels are the lever (E78b).** Held-out diagnostic accuracy after fine-tuning on 256 worlds whose training labels are corrupted at rate p (same worlds, patients, and prompts throughout; the test labels are always exact). Verified worlds (p=0) sit at the top; accuracy falls monotonically to the no-fine-tune base as the world becomes fully wrong (p=1). What world-time compute needs is label exactness, not task variety. Dots are per-seed values (3 seeds/level).

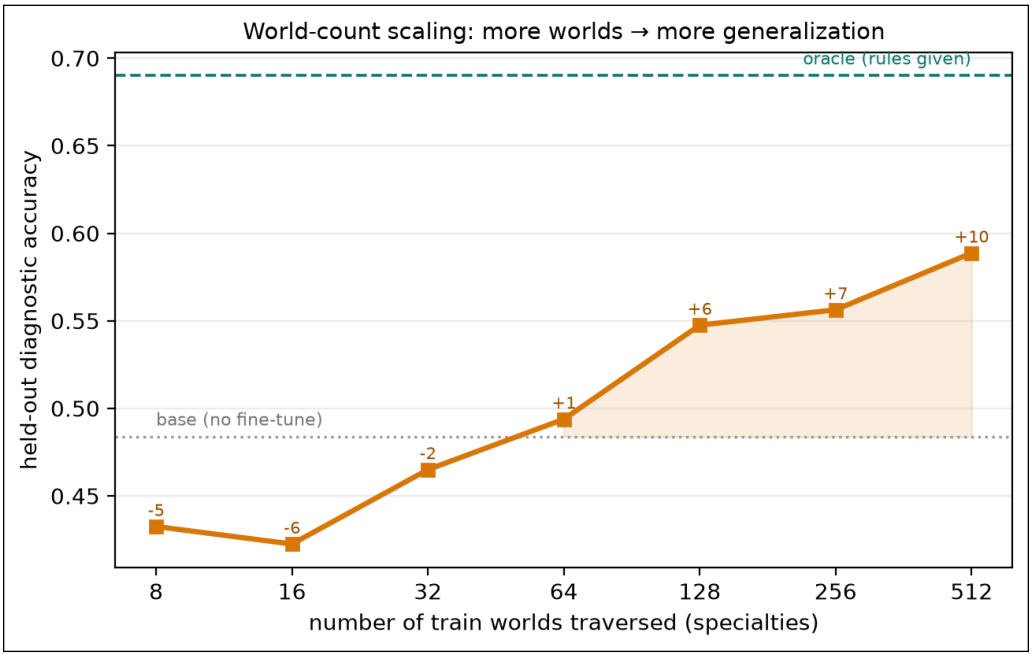

Figure 4: **World-count scaling (E76).** Held-out diagnostic accuracy of a 7B model vs the number of train specialties it traversed (hard family; test specialties fixed). Labels are the gain over the no-fine-tune base. Few worlds hurt; the gain rises monotonically and has not plateaued at 512, climbing toward the rules-given oracle. More worlds → more generalization.

unlike our synthetic worlds, where the rule is *stated* in the prompt, here the rule is *latent* and the model must induce it. Labels are exact: a predicted output grid is either right or wrong, with no judge. The train and evaluation tasks do not overlap, and each has a novel rule, so "generalize to held-out worlds" is the ARC challenge. We grow each task into many world variants with

Model	Synthetic p@1	Synthetic p@5	HumanEval p@1	HumanEval p@5
0.5B	0.17→0.27	0.38→0.55	–→–	–→–
1.5B	0.51→0.55	0.78→0.84	–→–	–→–
3B	0.78→0.70	0.85→0.91	–→–	–→–
7B	0.73→0.80	0.84→0.95	0.78→0.70	0.87→0.91

Table 3: **Coding world family (E77).** Sampled pass@1/pass@5 (n=5), base → world-time-compute (fine-tuned on 164 verified coding worlds), on synthetic held-out tasks (all sizes) and HumanEval (7B). World-time compute helps in-domain best-of-k at every size; on HumanEval it hurts greedy pass@1 but transfers positively at pass@5.

rule-preserving augmentation (the dihedral group × color permutations) and run two tests on a 4-bit Qwen2.5-7B. The per-task test-time-training recipe—fit a fresh LoRA on a task's augmented demonstrations—follows Akyürek et al. [2], who showed it is the effective approach on ARC (cf. induction/transduction program search [24]). Our contribution is not the recipe. It is to read the recipe as world-time compute on a verified world, and to add the corrupted-label ablation that pins down whether the exactness of the labels—not fine-tuning by itself—drives the gain. Figure 5 shows the regime end to end—world model → augmented, exactly-labeled rows → QLoRA fine-tune (frozen 4-bit base, trainable LoRA), with the two strict train/test splits—and Figure 6 reports the results.

(i) Cross-task transfer is weak. Train one LoRA on N real training worlds and evaluate it zero-shot on disjoint held-out worlds, and exact-match barely moves (3% base → 5% at 200 worlds). This is the boundary we expect: every ARC rule is novel by design, so a single shared policy has little to carry over. This matches E76/E77, where transfer only gets strong well above the world counts we can afford here.

(ii) Test-time training—world-time compute spent per world—works. For each held-out world we fit a fresh LoRA on that task's own demonstrations (its test pair strictly held out), then predict its test grid. This is the thesis in its purest form: compute spent learning a verified world buys generalization inside it. Exact-match climbs from 2% zero-shot to 8% (light) and 10% (heavy)—+8 points as we spend more world-time compute. The decisive mechanism check is the **corrupted-demonstration ablation**. Randomize the demo outputs (same grid structure, wrong labels), and the lift should vanish if the model is exploiting the exact labels rather than just gaining from fine-tuning. It does—the corrupt arm falls to 2% (Figure 6), at or below the 2% zero-shot floor. So the lift needs verified labels: the model learns each world's real rule, not grid statistics.

These are mechanism numbers, not a leaderboard entry: a single 7B at 4-bit with one seed over 40 held-out worlds, far below ARC solvers that add deep augmentation, voting, and program search. The claim is narrow, and we think it is the important one: world-time compute—spending compute to learn verified world models—lifts generalization on real, human-authored tasks, right where the "you generated it" critique bites hardest, with the corrupted-label ablation as the test of whether the labels' exactness is what earns that lift.

4.6 Across real domains and modalities—and where it stops (E80)

ARC is not a one-off. We ran the same per-world test-time-training protocol on two more real benchmarks from different fields, plus one different *modality* (Figure 7): **List Functions** [43] (60 hidden list→list functions induced from input/output pairs), **CLRS-Text** [26, 47] (29 classic algorithms—sorting, graphs, dynamic programming—each a hidden procedure to execute exactly), and **Bongard-RWR** [25, 32] (221 real-image visual concept-induction problems, via frozen DI-

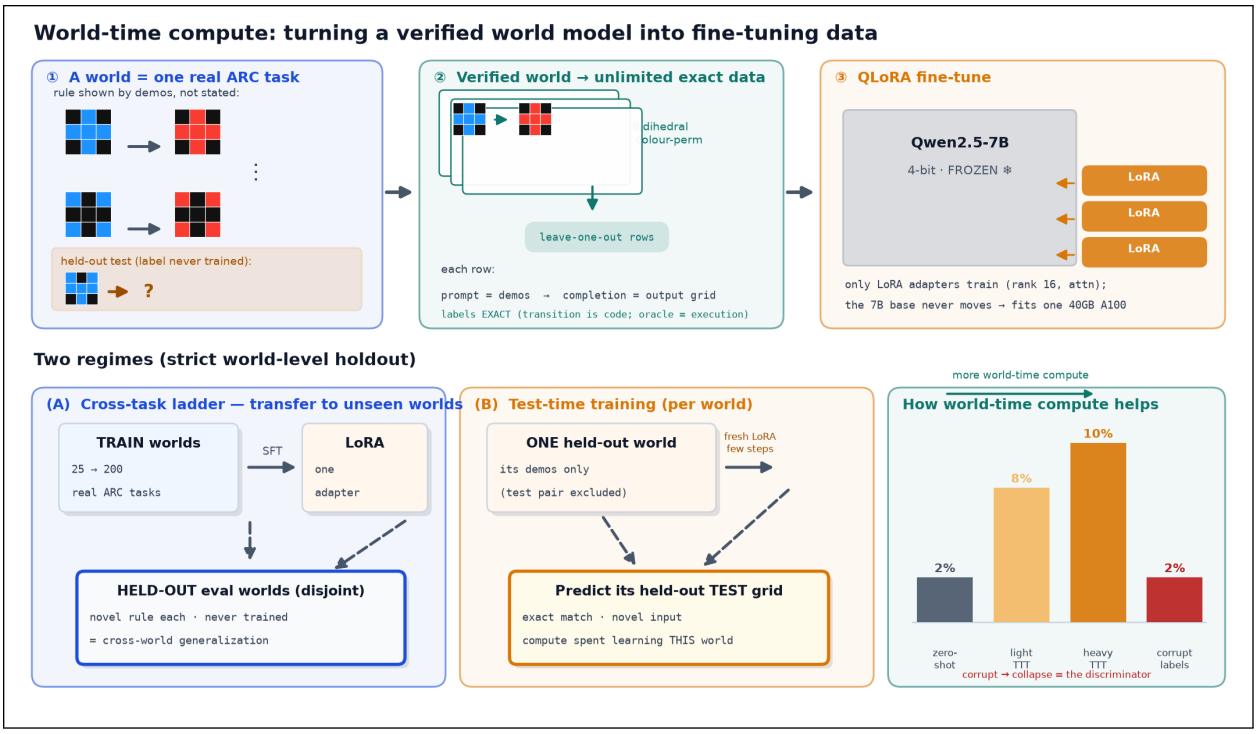

Figure 5: **How world-time compute fine-tunes on a verified world model (E80, ARC-AGI).** *Top:* each real ARC task is a world whose rule is shown by demonstrations, not stated; rule-preserving augmentation (dihedral × color) turns it into unlimited *exactly*-labeled rows (the transition is code, so the oracle is execution), which LoRA-SFT a frozen 4-bit Qwen2.5-7B—only the small adapters train. *Bottom:* two strict world-level holdouts—(A) the cross-task ladder trains on 25–200 worlds and is scored on *disjoint* held-out worlds; (B) test-time training fits a fresh adapter on a single held-out world's demos (its test pair excluded) and predicts that test. The payoff bars track exact-match against world-time compute, with a corrupted-label arm as the discriminator: if the lift survives label corruption, then the verified labels—not the optimizer—are not what does the work.

NOv2 features + a per-world head).

The trend replicates on every symbolic domain. Held-out exact-match *scales* with world-time compute and *collapses* to the floor once we randomize the demonstrations' labels: List Functions 35%→66%→69% (heavy 95% bootstrap CI over worlds [59, 77]), corrupt 3%; CLRS 9%→13%→17%, corrupt 4%; ARC 2%→8%→10% (CI [2, 20]), corrupt 2%. Same mechanism, three independent real benchmarks, and in each one the labels' exactness is what earns the lift—the discriminator from §5 holds everywhere.

Where it works: few-step reasoning. The size of the effect tracks the *depth of reasoning* an instance needs. World-time compute pays off most where the hidden rule is shallow—List Functions, a one-shot list transform, reaches 69%—and least where it needs long multi-step execution—CLRS, whose answers are long algorithm traces, tops out at 17%, the lowest of the three. This is the same boundary as the complexity cliff (E20; mitigated by composition in the companion framework paper [38]): verified and world-time-compute methods do best at tasks you can solve in *few reasoning steps* and fall off as the needed chain gets longer. It is also just where they beat data-hungry learned world models—on the short-horizon **MiniGrid DoorKey** planning task (§4.13) the verified model

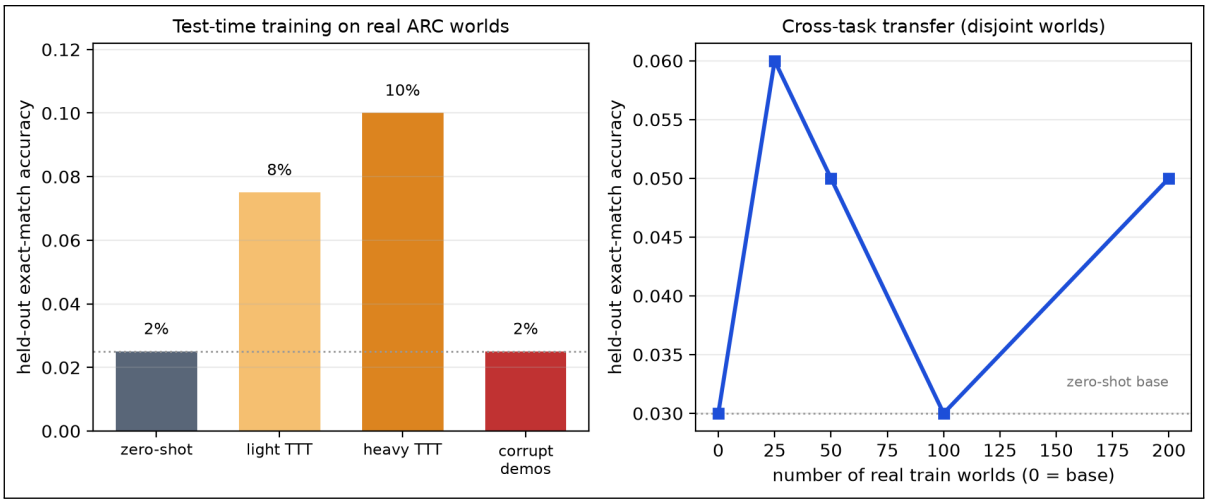

Figure 6: **World-time compute on real ARC-AGI (E80).** *Left:* per-world test-time training lifts held-out exact-match from zero-shot through light to heavy test-time compute; a corrupted-demonstration arm (wrong labels, same structure) shows verified labels are needed—the lift vanishes under corruption (2%). *Right:* cross-task transfer (train on N real worlds, evaluate on disjoint held-out worlds) is weak, as we expect from ARC's per-task novelty. Single 4-bit Qwen2.5-7B; mechanism, not leaderboard.

solves *zero-shot* while DreamerV3 needs $\sim 10^4$ interactions (9,640 to first solve; E65) and V-JEPA-2 cannot control at all.

The boundary: rich perceptual abstraction. Bongard-RWR is the honest negative: per-world test-time training over frozen visual features stays at chance (base 46%, heavy 48%, corrupt 52%; Figure 7, right)—no scaling, no collapse. When the model must *induce* the latent concept *from rich perception* instead of reading it from a symbolic input, a small head over frozen features gets no grip. World-time compute is a lever for symbolic, few-step reasoning, not a stand-in for perceptual representation learning—the same scope boundary the paper draws throughout.

4.7 Cross-world transfer on a real, human-authored domain (E84)

The real-data results above are per-world test-time training—the established recipe [2], not our contribution. The axis OpenWorld is built for is cross-world generalization: train once on a family of verified worlds, then generalize to held-out worlds. E83 found no such transfer across CLRS algorithms that share no skill. E84 tests the flip side of that—that a *coherent* family does transfer—on a real, human-authored domain (List Functions). We train one LoRA adapter on 128 disjoint held-in worlds, freeze it, and evaluate it zero-shot on 60 held-out worlds it never saw (3 seeds). Three arms share the same eval: **base** (no adapter), **cross-world** (adapter trained with exact labels), and **corrupt** (the same held-in prompts with randomized targets).

The cross-world adapter reaches 40% (95% CI [35, 45]) on worlds it never trained on, versus 6% ([4, 10]) for the corrupted-label control—a **+34-point** exactness-gated cross-world lift (CI [29, 39], which excludes zero), and it holds steady across seeds (42%, 43%, 36%; Figure 8). The instruction-tuned base scores 0% by exact match because it answers verbosely (chain-of-thought prose, not a bare list). The corrupt control soaks up exactly that format effect: it learns the output format from the same prompts, yet, with no correct rules, it gains only 6%. So the gap between cross-world and corrupt is *rule-induction transferring across worlds*, not formatting: an adapter that learned to infer list-function rules from one set of worlds infers new rules in worlds it never saw. This is the

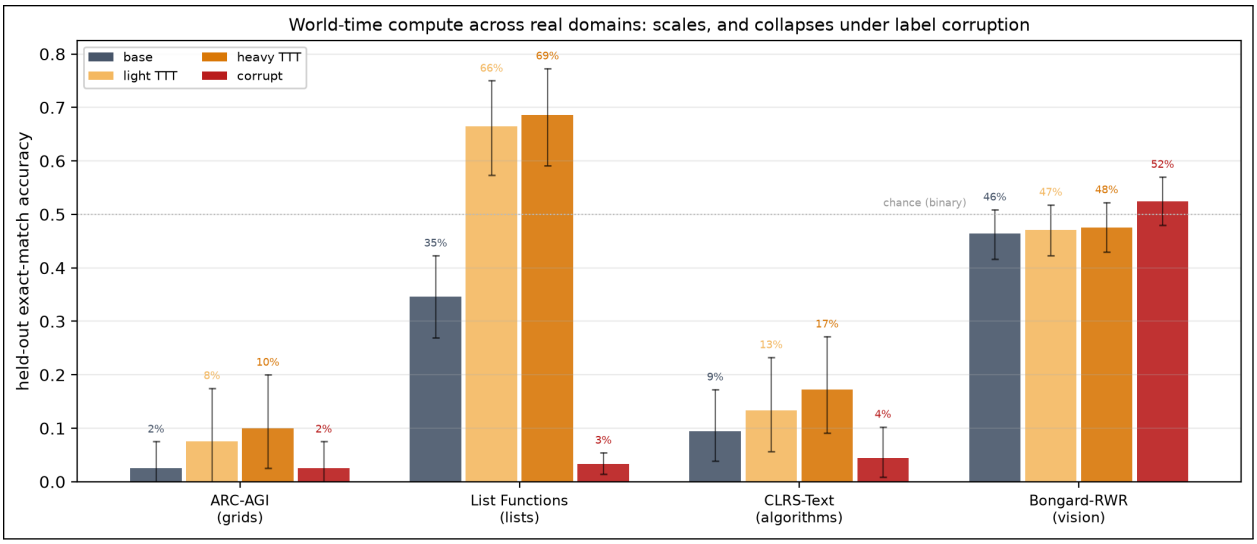

Figure 7: **World-time compute across real domains and modalities (E80).** Per-world test-time training (the same protocol everywhere) on real benchmarks: held-out exact-match for base → light → heavy world-time compute, plus a corrupted-label control; error bars are 95% bootstrap CIs over worlds. The three symbolic domains (ARC grids, List Functions lists, CLRS algorithms) all scale with compute and collapse under corruption—the lift needs verified labels. Bongard-RWR (vision, right) stays at chance: the boundary where the model must induce a latent concept from rich perception, not from symbolic input. (The ARC bars reproduce Fig. 6, left, shown here for cross-domain comparison.)

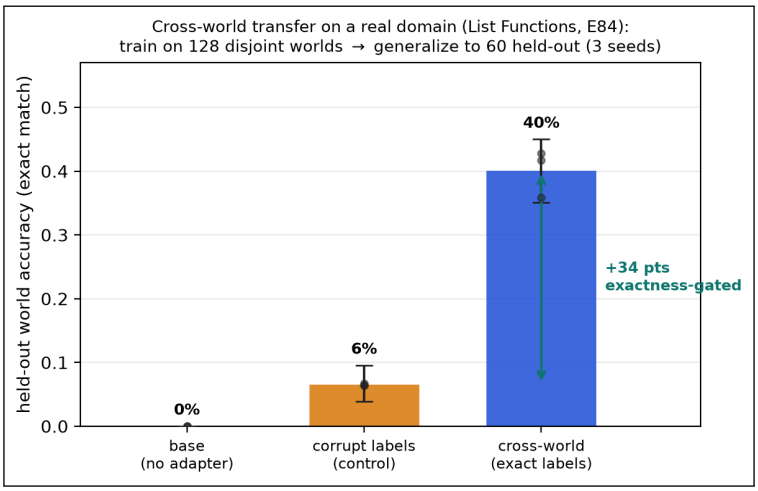

Figure 8: **Cross-world transfer on a real domain (E84).** One adapter trained on 128 disjoint List-Functions worlds generalizes to 60 held-out worlds it never saw (40%, 95% CI [35, 45]), while the same training with corrupted labels reaches only 6%—a +34-point exactness-gated lift (CI [29, 39]). This is the cross-world axis, on data we did not generate, that E83 (skill-disjoint algorithms) had left open.

novel axis—world-time compute as cross-world generalization—shown on a real, human-authored domain, the result E83 left open.

It scales with worlds traversed. The cross-world lift grows with the number of held-in worlds—from 29% at 16 worlds to a 49% peak by 64 worlds, then leveling off—while the corrupted-label control stays flat (Figure 9): a curve that rises then plateaus, for the novel axis, on real data.

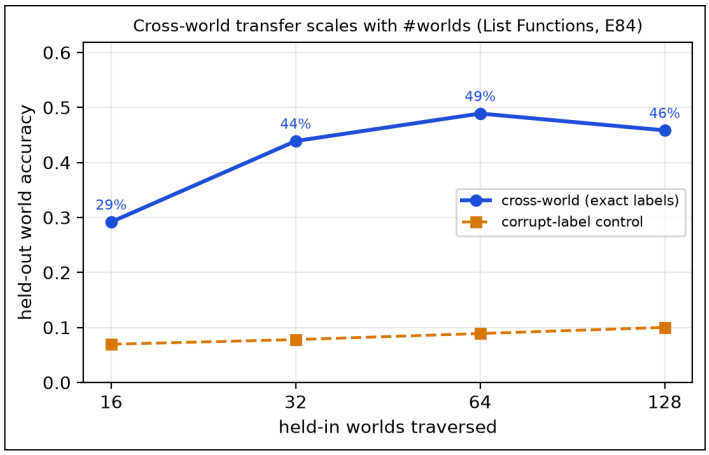

Figure 9: **Cross-world transfer scales with #worlds (E84, List Functions).** Held-out accuracy rises with the number of disjoint held-in worlds traversed (29%→49%), peaking near 64 worlds then leveling off; the corrupted-label control stays flat. (Single-seed ladder; the 3-seed headline of Fig. 8 is 40% at 128 held-in worlds.) More verified worlds → more transfer.

4.8 A descriptive regularity for world-time compute (E80)

The effect is regular enough across E80 to state as a *descriptive regularity*—not a predictive law. We fit it over 5 domains with single-seed runs and one out-of-sample check, the constants depend on the learner, and the fit is sensitive to which domains we pick (a sibling fit over a different subset is anti-predictive). We report the shape as a short summary, not a forecast. World-time compute follows a **saturating learning curve**, $\mathrm{acc}(c) = \mathrm{base} + (C - \mathrm{base})(1 - e^{-c/\kappa})$ (with compute-scale constant κ, distinct from the transition function τ), whose asymptote C tracks **identifiability** × **realizability** × **headroom** and which drops back to base without **exact labels**. We test it as a conjunction of falsifiable clauses over 333 worlds in 5 domains and three modalities (symbolic text, vision, and tabular); what travels is the form, not point predictions.

Clause 1 (scaling): a small probe predicts the asymptote. A light dose of world-time compute predicts the heavy-dose asymptote: within-domain Spearman is 0.98 (ARC), 0.90 (List Functions), 0.73 (CLRS), 0.77 (Bongard, vision), and 0.63 (tabular). A *leave-one-domain-out* fit—train the relation on the other domains, then predict the held-out one—reaches R^2=0.51, and the simplest model (light alone) beats adding parameters, which fits a real regularity rather than an overfit. One out-of-sample check is consistent, though not decisive: the fit on grids, lists, algorithms, and vision predicts per-world lift on a brand-new **tabular** domain it never saw, at Spearman 0.60—one held-out modality, not a broad forecast.

Clause 2 (no free lunch): the predictor must be a probe. We tried to predict the asymptote from zero-training signals (teacher-forced answer log-probability against the number of demonstrations). It fails—leave-one-domain-out $R^2 = -0.38$ (anti-predictive). In-context likelihood taps Bayesian updating; the TTT lift taps weight-space learnability, and the two pull apart. You cannot read the payoff off static signals; you have to spend a small dose of the actual process. This negative result sharpens the regularity.

Clause 3 (learner-invariance): not just neural networks. The same regularity holds for a *non-neural* learner—an enumerative program synthesizer over a list DSL, with search budget as

the compute axis: held-out exact-match rises and saturates (0.04 to 0.16 over the budget sweep), and corrupting the labels collapses it to 0.00 (no consistent program exists). Only which worlds are realizable depends on the learner (the DSL is a different hypothesis class than the network); the curve and the exactness gate do not. The regularity is about the verified-world learning problem, and it plays out the same way on gradient descent and on symbolic search.

Clause 4 (falsification): it predicts its own nulls. A regularity worth the name must say where it fails. On the 79 deliberately heterogeneous worlds (E71) cross-world transfer sits at the random floor; on ARC the cross-task ladder is flat; only coherent families transfer—diagnosis (E74) and, on a real human-authored domain, List Functions (E84, §4.7, a +34-point exactness-gated cross-world lift)—and inside the heterogeneous set a coherence-guided search still finds a generalizing subfamily (companion framework paper [38]). Here is a real-data probe of the same prediction (E83): train one adapter on 21 CLRS-Text algorithms and evaluate 8 held-out algorithms, and you get no cross-world lift—held-out accuracy does not rise above zero-shot—just as we expect for a set of algorithms (sorting, graph search, string matching) that share little skill. We report this as a weak null, though: the 7B base sits near the floor on these held-out algorithms, so the test has little room to move in either direction. One null we did not predict in advance, and report as found rather than forecast: cross-language-frame transfer (E81, §4.9) gives no lift, even though the exactness clause is consistent with it (matched frames beat corrupted). The lesson is the honest one—the regularity's form holds where we tested it, but a held-out language (or a CLRS algorithm) is its own skill, outside the family structure the regularity scores.

The reader's guideline: how many worlds to simulate. Because the curve saturates, the best number of simulated worlds is its knee. From the per-world compute curve, N_{90} (worlds for 90% of the asymptotic gain) is 3 for List Functions, 9 for ARC, and 15 for CLRS—*harder domains need more simulated worlds*, but only a handful, not thousands. Here is the scope: the curve's *form* seems to recur across the verifiable-world problems and learners we tested; the constants (κ, the depth cap) depend on the learner; and the predictor is cheap but not free—a small probe, which puts the truly hard part where it belongs, in choosing a representation that makes a world's rule identifiable.

4.9 Composites with varying rules: language frames and the invariant principle (E81)

Every world so far has one rule-set. But you can write the same computational *principle* under many rule-sets, and world-time compute should recover the principle that stays the same across all of them—a reference-frame view of generalization. Programming makes this concrete: a **composite** world is one programming problem, and its **sub-worlds** are that same problem written in 5 languages (Python, C++, Java, JS, Go)—each a verified code world whose transition is that language's tests running. A language is a *frame*: the same principle in different coordinates. We build 20 such composites from HumanEval-X. We then ask a simple question: does walking the other language-frames teach the frame-invariant principle well enough to solve a held-out frame? For each problem and target language we measure three arms: zero-shot (the base model on the held-out frame), *frame TTT* (60 steps of QLoRA on the other languages' prompt→solution pairs for that same problem, then evaluate the held-out frame), and corrupt (the same budget spent on mismatched-language pairs—the exactness control). We run two open-weight bases with different coding strength: Qwen2.5-Coder-7B (strong) and Llama-3.1-8B (weak).

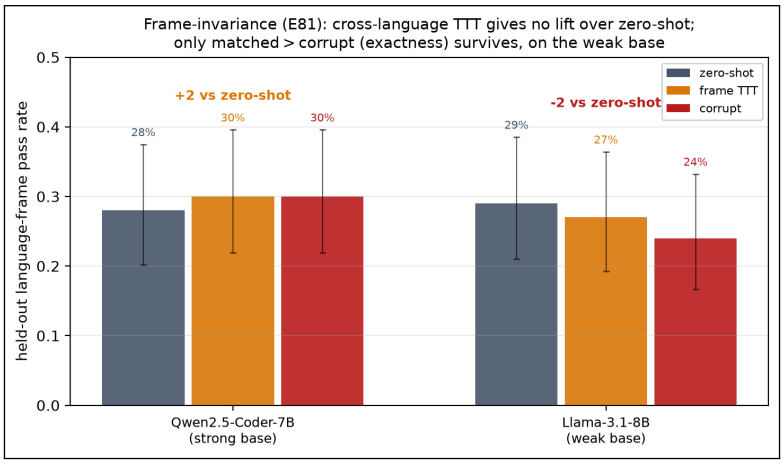

Figure 10: **Frame-invariance across programming-language frames is a negative (E81).** A composite world is one problem in 5 languages; frame TTT fine-tunes on the other languages and then evaluates a held-out language (n=100 problem×language holdouts per base). Cross-frame TTT does not beat zero-shot on either base: the strong coder stays flat (28%→30%) and the weak base drops (29%→27%); only the matched > corrupt ordering on the weak base (27% vs 24%) survives. We report this as a boundary of world-time compute. Error bars are Wilson 95% intervals.

This is the clearest negative in the paper, and we report it as a boundary (Figure 10). Cross-frame transfer does not beat zero-shot on either base. The strong coder barely moves, 28%→30% (within Wilson noise at n=100), and the weak base—where the headroom hypothesis predicted the largest gain—instead drops (29%→27%). The only structure that survives is a weak exactness ordering on the weak base: matched frames beat corrupted ones (27% vs 24%). But even matched frames fail to reach the zero-shot baseline. So this differs from the diagnosis-world (E74) and ARC (E80) settings, where world-time compute lifts held-out accuracy. Walking the language-frames of one problem does not teach a transferable principle: a held-out language acts like its own separate skill, and a few TTT steps on sibling languages disturb the model more than they inform it. The honest lesson is a limit on the reference-frame analogy: composites whose sub-worlds have genuinely different rule-sets (languages) are not automatically a single learnable frame, and exactness (matched > corrupt) is necessary but far from sufficient for transfer. The clean positive on this substrate comes not from cross-frame TTT but from the verified oracle used as a generator and backstop, which we isolate next (E82).

4.10 The hybrid loop: a verified world as generator and backstop (E82)

E81 shows that walking frames for free transfer fails on code; here the value of a verified world is its exactness, used directly. E82 isolates that. It puts all three routes (Table 7) in one open-weight, zero-human-label experiment on HumanEval-X Python, whose test suite is a perfect verifier. *Route 1 (tool as generator):* sample from the base model and keep only the solutions that pass the tests—exact training data with no labels (76/80 kept for Qwen, 66/80 for Llama). *Route 2 (amortize):* QLoRA-tune on those verified solutions and measure pass@1 again. *Route 3 (hybrid):* the amortized model generates, the tool verifies, and on a failure it retries up to 5 times. We run the same two bases as E81 over 40 held-out problems (Figure 11).

Three findings, one of them a caution. First, **amortization is not free**: self-distilling on verified solutions *helps* the strong base (80%→85%, +5 pts) but *hurts* the weak one (57%→50%, -7 pts)—the weak model's bootstrap set is thinner and noisier, so tuning on it disturbs the model

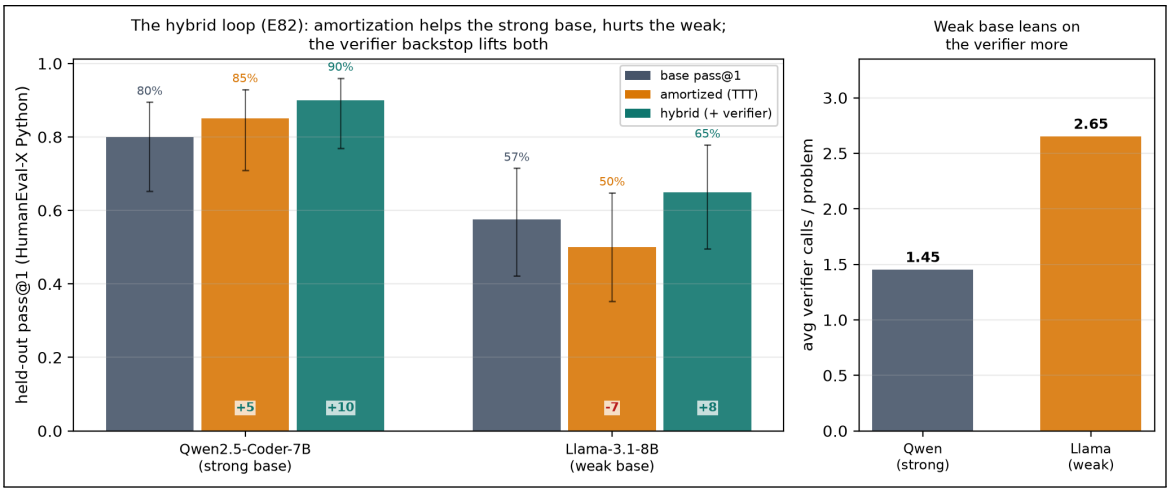

Figure 11: **The hybrid loop with open weights and zero human labels (E82).** Left: held-out pass@1 on HumanEval-X Python for base → amortized (QLoRA on self-generated test-passing solutions) → hybrid (amortized model + verify-and-retry, up to 5), with signed deltas vs. base. Amortization helps the strong base (+5) but hurts the weak one (-7); the verifier backstop lifts both (+10, +8). Right: the weak base leans on the verifier more (2.65 vs 1.45 calls per problem). Error bars are Wilson 95% intervals over 40 problems.

more than it teaches it. So the naive "smaller models gain more" prediction is *falsified* for pure test-time training on this task. Second, **the hybrid loop lifts both** above base (+10 pts for Qwen to 90%; +8 pts for Llama to 65%): the verifier backstop recovers what amortization alone puts at risk, and it is the robustly-positive route whatever the base strength. Third, the headroom signal that does survive is **verifier reliance**: the weak base calls the tool far more often inside the loop (2.65 vs 1.45 calls/problem). The practical reading maps onto the three routes: amortization (Route 2) pays off only when the base is strong enough to generate clean self-training data; the hybrid (Route 3) is the safe default; and a weaker model simply pushes more of the work onto the exact world. The exactness that E45 et al. buy, and that E81 found necessary-but-insufficient for transfer, is here *sufficient*—when the world is used as an oracle rather than a curriculum.

4.11 Against trained dynamics

The LLM next-state engine is a structural proxy; E12 adds real learned baselines trained on environment transitions (Figure 12). We trained an MLP and a 1-NN memorizer, each on $K \in \{100, 1000, 10000\}$ random-policy transitions, and held them to the same evaluation as the synthesized program. Two results stand out. First, sample efficiency: even at $K = 10{,}000$ the MLP reaches only 50% exact accuracy on the branch-covering probe suite, and 0% at 10× scale; the synthesized program scores 100% on both from zero transitions—its input is the rule text. Second, a measurement caution: at $K = 10{,}000$ the 1-NN memorizer completes 8/8 on-policy rollouts exactly while scoring only 30% on the probe suite. So on-policy rollout fidelity can be *memorized* without learning the rules. That is exactly why branch-covering probes and OOD evaluation, not rollout agreement alone, are the right instruments for dynamics quality.

E19 extends both axes (Table 11). A stronger learned baseline—delta-state target, double capacity, lr-decayed training—does no better than the original MLP (50% in distribution) and collapses in the same way out of distribution (0% at both 10× and 100×): the failure is not undertuning but extrapolation itself. Across a 1×/10×/100× scale ladder the synthesized program

is exact everywhere (100% at 100×); the LLM next-state engine is roughly scale-insensitive but unreliable across the board (∼40–50% at every scale)—it does not degrade OOD because it was never anchored to the distribution in the first place.

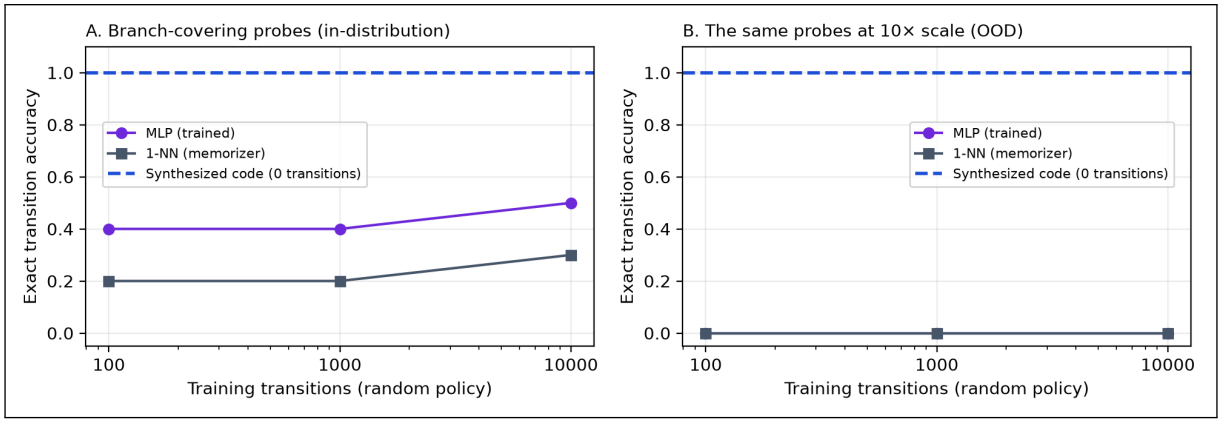

Figure 12: **Trained baselines vs. the zero-transition program (E12).** Exact transition accuracy of an MLP and a 1-NN memorizer trained on K random-policy transitions of the sprint world, on branch-covering probes in-distribution (A) and at 10× scale (B). The dashed line is the synthesized program, which uses no transitions at all. No trained baseline transfers out of distribution.

4.11.1 Induction on equal footing (E37)

A fair objection to the comparison so far is an *information asymmetry*: the synthesizer is handed the rules in natural language, while the learned baselines must infer the dynamics from sampled transitions. E37 removes it (Table 4). The LLM sees only observed (state, action, next) transitions—no rule text—and must induce a `transition()` program, accepted only if it reproduces the observed transitions; we then hold it to the same in-dist and 10× OOD probe suites as the MLP and 1-NN trained on the identical data. Two findings result. First, on equal information, code induction beats statistical learning out of distribution by a wide margin: 0.43 versus 0.00 (MLP) and 0.00 (1-NN). The advantage is *structural*: induced-code accuracy is the same in-distribution and at 10× scale (0.43 vs 0.43, on every replicate), because the model induces symbolic rules that extrapolate, whereas the learned baselines fit magnitudes and collapse to zero OOD on every replicate. So the compounding/extrapolation advantage is not an artifact of being handed the rules. Second, induction is *imperfect*: the 7B model recovers only 0.43 of the probe behavior from traces (it misreads the subtle `debt`-dependent interaction), well below the rule-text anchor's 1.00. The gap between 1.00 (rules) and 0.43 (traces) is what the specification contributes—neither nothing nor everything—and it measures the asymmetry the headline comparison carried. Larger budgets do not close it here: the reachable transition set under a random policy saturates, so K=100 and K=1000 give the inducer essentially the same distinct examples.

Is the induction ceiling a capability limit? (E38) If ∼0.43 reflects a weak 7B generator, a stronger model should close the rules-vs-traces gap. It does not (Figure 13). Re-running E37's trace-induction across 3 generators from three families holds the gap roughly fixed: 0.43 for qwen2.5:7b, 0.33 for the larger qwen3-coder:30b, and 0.43 for the reasoning model gpt-oss:20b—none coming near the rule-text anchor's 1.00, and every model exactly scale-invariant (in-distribution accuracy equals 10×-OOD accuracy). Notably the 30B coder reproduces the observed traces best (∼0.9 of training transitions) yet generalizes no better; it fits the shown examples without recovering the

Method	Information given	In-dist.	10× OOD
Code synthesis (rules)	rule text, 0 transitions	1.00	1.00
Equal information: 1000 random-policy transitions, no rule text			
Code induction	traces only	0.43	0.43
MLP	traces only	0.40	0.00
1-NN memorizer	traces only	0.17	0.00

Table 4: E37, equal-information induction: exact probe accuracy when the LLM must induce dynamics from transitions alone (no rule text), against the MLP and 1-NN on the same data, plus the rule-text synthesis anchor. Code induction is imperfect but scale-invariant (in-dist = OOD); the learned baselines collapse out of distribution; the rules-vs-traces gap measures what the specification buys.

latent rule. So the ceiling is an *identifiability* limit of the data, not a capability limit: random-policy transitions do not pin down the subtle interaction, so no generator can recover it from them. Closing the gap needs informative transitions (active experiment design), not a bigger model. (deepseek-r1:14b is excluded: its reasoning-trace length made trace-induction impractical on the local hardware.)

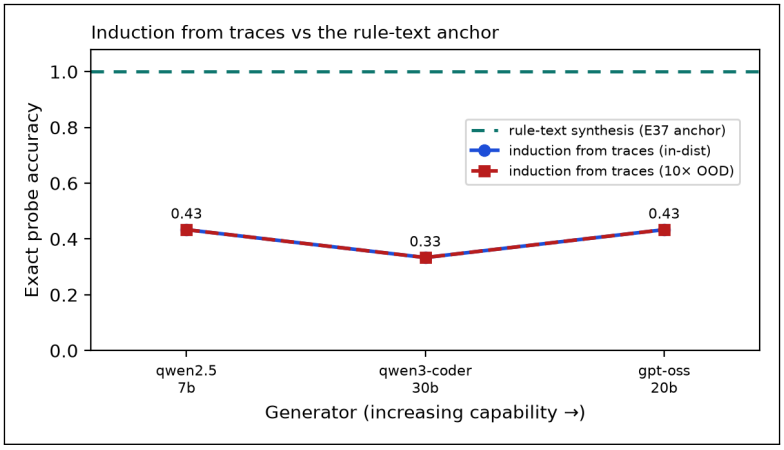

Figure 13: **Induction does not scale with generator capability (E38).** Trace-induction probe accuracy across three generator families; the rule-text anchor (E37) is 1.00. The gap is roughly flat in model size and exactly scale-invariant (in-dist = OOD)—an identifiability limit of the data, not a capability limit.

4.11.2 Acting closes the gap that scale cannot (E43)

E38 names the bottleneck (identifiability, not capability) and points to the fix: *informative* transitions rather than a bigger generator. E43 tests that fix directly. We cast dynamics recovery as version-space elimination over a candidate family of 108 sprint rules that set the unknowns (ship's debt increment, the divisor k in the subtle bugs $+= \lfloor \text{debt}/k \rfloor$ interaction, and the fix/refactor magnitudes); a candidate is dropped the moment its predicted next state disagrees with an observed transition. We compare three probing policies on 5 hidden true rules: a *passive* random policy (the E38 setting); an *active* policy that picks the action that most splits the surviving candidates and tie-breaks toward `ship` to build the debt that makes k observable; and a *clairvoyant* reference that already knows the rule and greedily removes the most candidates per step.

Acting wins decisively (Figure 14). The active policy identifies the exact rule in 11.2 transitions on average, versus 14.7 for the passive policy. And crucially, passive observation never resolves 2/5

of the hidden rules within the budget—exactly the high-k rules whose interaction only shows up at debt the random policy rarely reaches. This is the E38 ceiling seen again as a plateau (Figure 14, left): no amount of passive data crosses it. The active policy, knowing nothing about the rule, matches the clairvoyant reference within a step on average (11.2 vs. 11.6) and sometimes beats it—the non-myopic "build debt now to disambiguate later" move is exactly what the greedy reference, which optimizes per-step, fails to make. The lesson sharpens E38's: when traces underdetermine the dynamics, the leverage is in *choosing* the transitions, and a world model the agent can act inside is what makes that choice expressible.

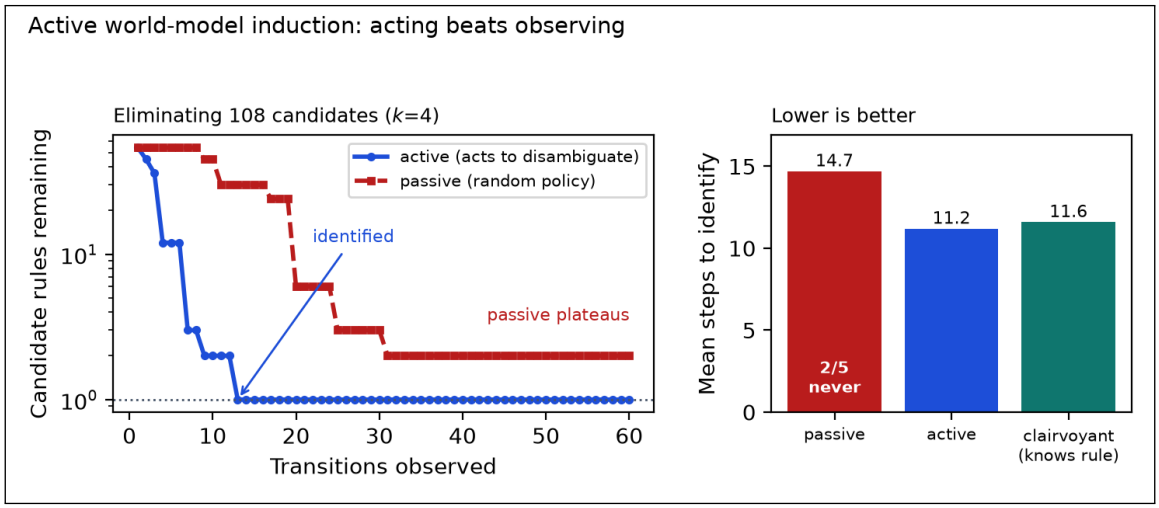

Figure 14: **Acting beats observing for rule identification (E43).** Left: candidate rules remaining (log scale) as transitions accrue, for a hidden rule the passive policy never resolves—active elimination drives 108 candidates to one while passive observation plateaus. Right: mean steps to identify across 5 hidden rules; the active policy (knowing nothing) matches the rule-knowing clairvoyant reference, while the passive policy is both slower and leaves 2/5 rules unresolved.

Further world models on the same primitives. Beyond induction, the framework can express many more world models that show off its range rather than carry the central argument: a non-enumerable version-space database (E46), path integrals over learning trajectories (E49), next-token world models with exact length generalization (E45), composite corporate (E48) and startup-growth (E51) worlds, a tree-of-thoughts brain simulator with real persistent memory (E58, E59), and the full perceive→world→emit→act boundary (E60). These are the subject of the companion framework paper [38].

4.12 From fidelity to decisions: planning

World models exist to improve decisions; E22 closes that chain (Table 5). A model-predictive planner runs exhaustive depth-3 lookahead inside its world model and then executes the best action in the ground-truth environment. Planning through the verified synthesized program scores 7.5—the same as planning through the ground truth itself (7.5), as bit-exactness guarantees—against 5.0 for a reactive heuristic and 3.8 for random actions, with the whole 12-step episode planned in 0.03 s. Planning through the LLM next-state engine is held to depth 2 by cost (86 s per episode) and scores 0.0—*worse than no model at all*: lookahead through hallucinated transitions steered the agent away from productive actions entirely. We note one caveat: the LLM engine falls back to a no-op when a reply does not parse as a next state, so this score mixes together genuinely

hallucinated transitions and unparseable outputs; the script now logs the parse-failure rate so the two can be separated on a re-run, but the qualitative conclusion—planning through an unverified model is worse than not planning—does not depend on that split. Fidelity and throughput are not abstract virtues; they are the difference between planning that helps and planning that harms.

Policy	Episode score	Planning time (s)
Lookahead d=3 via synthesized code	7.5	0.03
Lookahead d=3 via ground truth (bound)	7.5	0.00
Lookahead d=2 via LLM next-state	0.0	85.85
Reactive heuristic (no model)	5.0	0.00
Random policy (5 seeds, mean)	3.8	—

Table 5: E22: model-predictive lookahead on the sprint world, plans executed in the ground-truth environment. Value function: shipped − bugs − 0.5·debt after 12 steps.

4.13 Crossing species: verified vs. trained vs. perceptual world models (E63, E65, E67)

The comparisons so far pit verified code against an LLM next-state proxy; a model-based-RL reader asks how it fares against world models trained on transitions, judged on task return. Across 6 locally-runnable world models on 2 symbolic domains under one battery (K=10,000 transitions, 5 seeds; Table 6), the result is uniform: the verified code model is exact in- and out-of-distribution, rolls out bit-exactly from zero transitions, and plans optimally (return 7.5) at 2,250,984 steps/s, while every learned model drops to 0.00 exact accuracy at 10× OOD and plans at or below model-free control—sometimes negative (planning through a trained model is worse than not planning, as lookahead drives the agent into unseen states; E61). Perceptual world models—DreamerV3 [19], MuZero [37], IRIS [27], DIAMOND [3], Genie [6], Sora [31], V-JEPA [5]—are a different species (they predict observations or embeddings, not symbolic state), so we compare them on *properties* (Table 6, below), not a fabricated shared number.

World model	Train data (transitions)	Probe in-dist	Probe 10×OOD	Rollout exact	Control return	Audit.
verified code (CWM)	0 (rules)	1.00	1.00	1.00	+7.5	✓
1-NN	10,000	1.00	0.00	0.77	+5.7	–
tabular	10,000	1.00	0.00	0.77	+5.7	–
linear	10,000	0.46	0.00	0.19	-7.4	–
koopman	10,000	0.75	0.00	0.44	+5.6	–
MLP	10,000	0.70	0.00	0.19	-2.6	–
LLM next-state†	rules	–	–	0.00	+0.0	–

Table 6: **World-model bake-off (E63).** Every world model we can run locally on shared symbolic tasks (5 seeds, K=10,000). Verified code is the only model exact, OOD-robust, optimal-for-control, auditable, and zero-data; perceptual models are a different species (perceptual, not symbolic), positioned against OpenWorld in the introduction.

Where the species can meet (E65 MiniGrid, E67 cartpole). On benchmarks where learned models are strongest—image-based MiniGrid DoorKey and pixel cartpole-swingup, each with OpenWorld's transition validated bit-for-bit against the real environment—the picture holds on the

learned models' own terrain. OpenWorld solves DoorKey zero-shot (a 11-step optimal plan, no data), where DreamerV3 needs ∼10k pixel interactions just to succeed once; on cartpole it solves 100% of episodes with zero data via CEM-MPC, while a frozen-perceptual V-JEPA-2 fails through every controller we tried (a control failure, not a perception failure). These are scope properties, not contests—handed the dynamics, the verified model need not learn them; the full three-species tables are in the companion framework paper [38]. The boundary is consistent: verified world models win where the *reasoning*, not the perception, is hard and the horizon is short (DoorKey), and stall where rich perceptual abstraction is the task (Bongard-RWR, §4.6).

4.14 Scope boundaries and oracle-free verification

Three round-3 experiments chart where the paradigm holds and what to do at its edge.

The complexity cliff (E20). Parametric worlds with R interacting rules (conditions reading pre-action state, summed effects, clamping) were synthesized from rule text at $R \in \{4, 8, 12, 16\}$ (Figure 15). Mean probe accuracy falls from 1.00 at R=4 to 0.53 at R=8, 0.25 at R=12, and 0.15 at R=16: monolithic single-function synthesis with a 7B generator degrades sharply once roughly eight coupled rules must be written at once. This is the paradigm's present boundary at this model scale, and it motivates compositional synthesis—one expert per rule, as in PoE-World [33]—as the structural fix.

Stochastic dynamics (E21). Threading a seed through the state (`rng_seed` in, derived seed out) extends verified synthesis to stochastic worlds without giving up replayability. On a queueing world with a declared 0.3 arrival probability, accepted programs (67% of attempts) matched the declared distribution to within 1.2 percentage points over 2,000 seeded transitions, kept the deterministic bookkeeping perfect (100%), replayed bit-exactly, and in fact matched the hand-written stochastic oracle bit-for-bit (100%).

Oracle-free error detection (E23). Practitioners lack the hand-written oracles our evaluations use; they can, however, synthesize the dynamics several times. Across 30 stored programs and 900 program–probe pairs, disagreement with the ensemble majority caught ground-truth errors with 100% precision and 100% recall—the majority vote rebuilt the oracle exactly on these worlds—and a program's agreement rate ranked its true accuracy perfectly (Spearman $\rho = 1.00$). The caveat is structural: majority-as-oracle fails under correlated errors (e.g., every model misreading the same ambiguous rule), so ensemble disagreement is a cheap screen, not a proof; it composes naturally with the invariant and critic gates.

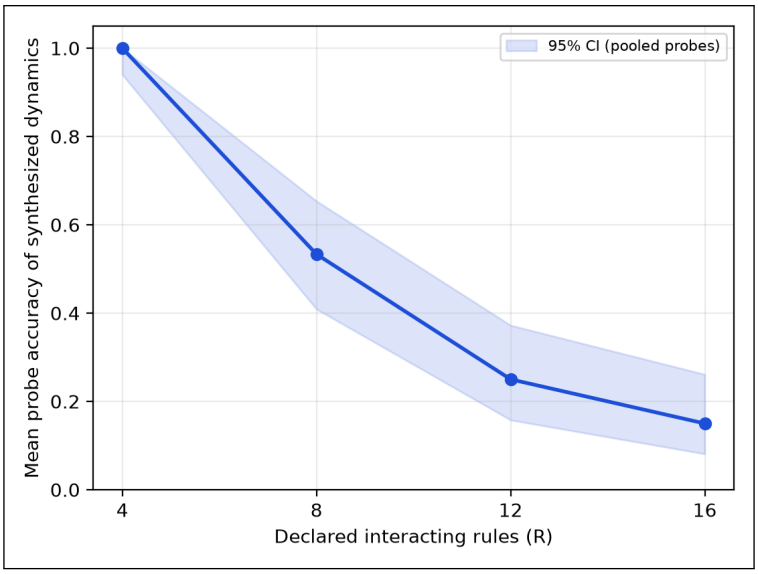

Figure 15: **The complexity cliff (E20).** Probe accuracy of 7B-synthesized dynamics versus the number of declared interacting rules; three syntheses per point, shaded band is the 95% CI over pooled probes. Monolithic synthesis is reliable at R=4 and unreliable past R≈8.

5 Discussion and Limitations

Interpretation. The experiments measure a structural trade-off. Learned world models spend training compute to cover pixel-native breadth, and they pay for it with compounding error, opacity, and frozen values. Verified code synthesis buys exactness, speed, OOD transfer, and auditability, and it pays at the boundary of what you can state symbolically. Inside that boundary the results are one-sided: a laptop-scale local model, run as plan–generate–verify, builds simulators that are *perfect* where learned-style baselines are only approximate. The ablations license a plain workflow: state the rules precisely; verify them with invariants; test the accepted artifact against what you meant; then tighten and recompile. Each gate buys measurable correctness, and the artifact stays a diffable program at every stage.

Is world-time compute just fine-tuning? Mechanically, yes: there is no new optimizer or architecture, only LoRA-SFT. The claim is about the *data source*, not the optimizer. Ordinary fine-tuning needs a labeled corpus—scarce, costly, and sometimes wrong; world-time compute instead fine-tunes on trajectories from a *verified* world model, which makes unlimited examples and labels them *exactly* for free, because the transition is code and the oracle is running it. In one line: a correct world model turns compute into unlimited correct training data. Is the "verified" qualifier really load-bearing—or just decoration on plain SFT? That is an empirical question with a clean discriminator, the **corrupted-label ablation**: randomizing the labels keeps the fine-tuning and the world's structure but destroys their correctness, so if accuracy does not change, the framing adds nothing. It does collapse to the no-fine-tune floor: the corrupted-demonstration arm on real ARC worlds falls to 2% against the verified arm (§4.5, Fig. 6). So the contribution is conceptual, not algorithmic, and only two things earn the approach its name over "SFT with extra steps": (i) this verified-label dependence, and (ii) cross-world transfer—training on a family of worlds lifts accuracy on held-out worlds never fine-tuned on (E74, E76), which same-world fine-tuning cannot do. Where neither holds—heterogeneous worlds with no shared skill (E71, E73), or a benchmark of

per-task-novel rules where cross-task transfer stays weak (ARC, §4.5)—it is, fairly, just expensive fine-tuning, and we say so. This puts world-time compute squarely in the self-training lineage—STaR [50], ReST and ReST-EM [16, 40], expert iteration [4], rejection-sampling fine-tuning [49], and learning from verifiable rewards [21]—all of which retrain a model on its own outputs filtered by a correctness check. Two differences are the contribution, not the loop: the verifier is a *full world model with dynamics*, so the training unit is a verified trajectory (a state-action-next-state transition), not a checked final answer; and the target is a different axis of generalization—held-out worlds in a family, rather than the same task improved in place. In those terms the corrupted-label ablation is exactly a verifier-reliability study: the failure mode (training on confidently-wrong self-labels) that self-training fears most, here ruled out by construction.

The trajectory unit, measured (E85). We test the first difference—the unit is a trajectory (s, a, s'), not an answer—directly, on synthetic iterated worlds: one cross-world adapter trained on verified single-step transitions vs. another trained on $(s_0 \to \text{final})$ answer pairs from the same worlds (3 seeds). At the trained horizon the trajectory unit wins (70% vs. 58% for the answer unit, +12 points)—the unit, not the loop, carries the signal. Rolled out to a horizon *longer* than trained, the answer unit catches up (75% vs. 68%)—the model's per-step predictions compound, the very error mode verified code avoids. So the trajectory is the better training signal, but composing it at inference brings compounding back on the learned side.

When does it pay off? Few-step reasoning. Pulling the results together (§4.6) gives a single operating regime. Verified world models and world-time compute win where a task boils down to a few reasoning steps over a state that can be *told* symbolically: shallow rule induction (List Functions, the largest lift), small grid transforms (ARC), and short-horizon planning (MiniGrid DoorKey, solved zero-shot while DreamerV3 needs 10^4–10^5 interactions [19]). Performance drops along two axes—as the required reasoning *lengthens* (CLRS algorithm traces, the lowest of the symbolic domains; and the E20 complexity cliff past ∼8 composed rules), and as the latent rule must be induced from rich perception rather than read from symbolic state (Bongard-RWR vision, flat at chance). This is not a hedge but the shape of the contribution: a verified, auditable, zero-data lever that is strongest exactly where today's data-hungry learned world models are weakest—short, specifiable reasoning—and that cleanly hands the long-horizon, perception-heavy regime to them.

Three routes to use a world model. A verified world model is a reusable artifact, and it is useful whether or not you ever fine-tune; world-time compute is one of three ways to spend it (Table 7). *Route 1—as a tool:* serve the world and call it—plan through it, ask for exact next-states, or use it as a verifier (no training; exact, auditable; §4.12, §4.13). *Route 2—per-world test-time training:* distill a single world's exactly-labeled trajectories into weights with QLoRA, amortizing that world's skill for tool-free inference (§4.5). *Route 3—world-time compute:* traverse a *family* of verified worlds and fine-tune to generalize to held-out worlds (§4.3, §4.8). The *hybrid* combines them: use the tool to *generate* exact trajectories, train to *internalize* them, and keep the exact tool as a fallback for high-stakes queries—the tool bootstraps the data, training amortizes it, and exactness is recoverable on demand.

Value alignment as an open specification. The dial experiments ground the alignment claim: because objectives are declared artifacts, the welfare–fairness frontier is operational—an operator moves along it in real time, and a tuner searches moral configurations jointly with world design,

Route	Exact?	Training	Tool at inference	Generalizes to new worlds
1. Tool	yes	none	yes	n/a (use any world)
2. Per-world TTT	approx.	QLoRA, one world	no	within the world
3. World-time compute	approx.	QLoRA, many worlds	no	yes (across the family)
Hybrid	on fallback	QLoRA	only when unsure	yes, with exact backstop

Table 7: **Three routes to use a verified world model**, plus the hybrid. Need exactness/audit → Route 1; amortize one world → Route 2; generalize a family → Route 3; best of both → Hybrid.

surfacing the whole manifold of value settings that solve a task rather than a single frozen compromise. Two qualifications, owed to the philosophical review. First, the openness reaches only as far as the expressible structures: a weighted sum is act consequentialism, and the companion framework paper [38] shows that aggregator choice, side constraints, and theory uncertainty are distinct moral objects the framework must (and now does) represent rather than approximate with weights. Second, open specification *manages* the specification trap [42] procedurally rather than resolving it: the trap moves from frozen weights to the operator's hand on the dial, and which hand that should be—justifiability to those affected, in the contractualist's sense [36]—is a question no architecture settles. The tuner's solving manifold is at best a crude contractualist surface: the set of configurations that meet every stakeholder's declared minimum.

Limitations. (i) Scope: symbolic-state worlds; pixel-native domains stay the territory of learned models—the comparison here is within the symbolic regime, and neither the LLM proxy nor the numpy baselines is a trained Dreamer. The headline comparison also hands the synthesizer the rule text; E37 shows the structural OOD advantage survives removing that asymmetry (induction from traces alone), but induction is then imperfect (0.43 vs the rule-text 1.00), so part of the headline margin reflects the value of an explicit specification, not the representation alone. (ii) Complexity: monolithic synthesis with a 7B generator is reliable at four interacting rules and unreliable past roughly eight (E20); the headline worlds sit below that cliff, and richer worlds will need compositional synthesis [33] or larger generators. (iii) Scale: twenty repair tasks and three handwritten worlds; paired tests sharpen the comparisons, but the benchmark is still an archetype suite, not HumanEval [7], 3B+ agents saturate it, and the classic defect archetypes are plausibly present in pretraining data: agent solve rates should not be read as measurements of debugging skill—the world-model claims are unaffected, since the environment runs exactly regardless of what the agent has memorized. (iv) Synthesis replicates across three model families (E16), but the judge, critic, and repair-agent roles stay Qwen-only. The synthesis, judge, and repair experiments run on one consumer machine under the stated quantizations; the fine-tuning experiments (E74, E78, E80–E83) instead run on a single cloud A100 (4-bit QLoRA for the 7B and larger models), which we state plainly so the consumer-hardware framing is not read as covering the training runs. (v) Judges: selection accuracy and order-consistency are measured but imperfect, the pooled selection margin is marginal ($p = 0.078$) and costs $\sim 4\times$ inference, and rubric grading showed compression; judge verdicts should gate, not replace, programmatic verification. (vi) Ensemble self-checking (E23) reached perfect precision/recall here but is structurally vulnerable to correlated errors across generators reading the same ambiguous rule; it is a screen, not a proof. (vii) The sandbox is best-effort isolation against accident, not an adversarial security boundary (generated patches twice defeated in-process timeouts before the fork-based runner; see the repository history). (viii) The moral machinery, even broadened, evaluates declared values; whose values are declared—and the legitimacy of the declaring hand—is a procedural question outside any architecture [36], and the parliament's credences are themselves operator-set dials. (ix) Several experiments were

redesigned after pilots and four internal review rounds (E3, E5/E6, E11–E27); pilot saturation results are reported alongside the final protocols, and all reviews ship in the repository. (x) Multiple comparisons: across 79 experiments we report many p-values and rank correlations and apply no family-wise or false-discovery correction; with this many tests the borderline results (p between 0.05 and 0.10—notably the judge margin and the rubric-paraphrase correlation) should be read as exploratory, and only the large deterministic effects (exact-match rates, OOD collapse, the complexity cliff) hold up under the multiplicity. (xi) Variance: the deterministic claims are exact-match against oracles and need no error bars, but several *stochastic* (LLM) headline quantities are point estimates from small, single-seed samples (e.g. the E1 divergence step, n=3, one world) with no across-seed variance; we add the pooled multi-world view (E11) where it matters, but per-model probe accuracies should be read as estimates, not tight measurements. This caveat extends to the headline world-time-compute results: the E74 model-size scaling is a single fine-tuning seed per size, and the real-data ARC/CLRS arms are single-seed at $n\approx$30–40 with CIs that touch the floor, so the size-trend and the weaker real-data domains should be read as exploratory; for ARC we report a 2-seed re-run—heavy 11% (seeds 10%, 12%) vs. corrupt 4%—so the verified-vs-corrupt gap holds across seeds, though absolute scores stay mechanism-scale. (xii) Two illustrative studies are true *by construction*: E59's architecture lift follows arithmetically from a modeled fixed-capability backbone, and E60's routing gap over exact-key lookup is guaranteed by the comparison design (we add a fair lexical baseline and report the modest gap over it); both are flagged in place as engineering demonstrations, not measured effects. (xiii) Porting to real data surfaces two validity conditions: the input representation must encode per-world structure (a ProteinGym port describing each mutation only by global biochemical deltas made every assay-world identical and flattened the world-count curve until per-world local-sequence context was restored), and faithful data loading—e.g. the wild-type sequence and one-based mutation indexing—is the dominant correctness risk. We treat both as the first checks any real-domain port must pass. (xiv) The trajectory-vs-answer distinction is isolated in E85 (above) on synthetic iterated worlds; porting that head-to-head to a multi-step real domain (where trajectory and answer diverge most) remains future work.

6 Conclusion

A training-free symbolic world model—synthesized as code by a local model and verified before acceptance—is bit-exact where learned-style dynamics compound error, and plans *as well as the ground truth itself* where planning through a hallucinating model is worse than no model. And beyond serving as a simulator to plan in, the same verified worlds are training signal: traversing many world models of a domain lifts a model's generalization to held-out worlds, scaling with the number of worlds traversed—world-time compute, a training-time analogue of test-time compute. Its boundary is measured just as sharply: monolithic 7B synthesis fails past roughly eight interacting rules. The world-model field's path has been to scale parameters until hallucination is suppressed; these results argue for an orthogonal path—make the model a program, verify the program, and keep the values dialable. Next steps: compositional synthesis to cross the complexity cliff [33], the coding world at standard-benchmark scale, hybrid perception-to-symbol pipelines, and judges that propose—not just select—world repairs.

Declaration of generative AI use

During the preparation of this work the authors used *Claude 4.8* (Anthropic, via Claude Code) to implement the OpenWorld framework and experiment scripts, run the experimental campaign,

generate figures and tables, and draft and edit this manuscript and the experiments, under the authors' direction. The authors reviewed and edited the content and take full responsibility for the content of the publication. All experimental results are produced by the released deterministic scripts (not by generative-AI estimation), and no AI tool is listed as an author, since such tools do not meet authorship criteria.

Data and code availability

All code, the benchmark suite, experiment scripts, raw result JSON, and this manuscript's build are available at `github.com/quome-cloud/openworld`. All data are synthetic; no human-subject or patient data were used.

Author contributions

J.S. conceived the study, directed the implementation and experiments, and reviewed the manuscript. I.S., J.R., M.O., C.O., C.K., A.E., M.B., R.T., J.T., and M.G.F. contributed to the research and edited the manuscript. Many of the authors are active contributors to the OpenWorld open-source project: they ran their own simulations with the framework and provided feedback that improved the product. All authors reviewed and approved the final manuscript.

Competing interests

The authors declare no competing interests.

Funding

This work received no specific external funding.

A Index of experiments

The experiments E1–E85 are introduced across the Results section in thematic rather than numeric order; this index lists each by number with a one-line description and the section, figure, or table where it appears, as a navigation aid. Numbers are non-contiguous: E14 and E53 are retired (absorbed or cut) and do not appear in the text; E28–E29 are described in the Experimental Setup but their dedicated results section was cut; and E66, E69–E73, E75, E78 (its E78b variant *is* used), and E79 were not used (a superseded probe or a negative result excluded from the paper). The world-time-compute family E74–E85 (E80 spans several real-data domains and the descriptive regularity) carries the central result.

Table 8: Index of experiments E1–E85. E14, E53 retired and E66/E69–E73/E75/E78/E79 unused (absent from the text); E28–E29 are described in the Experimental Setup but their results section was cut.

E#	Description	Where
E1	Head-to-head symbolic vs. learned dynamics on the sprint world	Tab. 1
E2	Synthesis reliability: five compile attempts per world	Tab. 9
E4	Head-to-head engine comparison, primary sprint result	Tab. 1

continued on next page

E#	Description	Where
E9	Configuration-search strategies on triage auto-tuning	Tab. 12
E10	Head-to-head primary comparison, throughput and accuracy	Tab. 1
E11	Multi-world fidelity replication, eight scripts per world	Tab. 2
E12	Trained MLP/1-NN baselines vs. the zero-transition program	Fig. 12
E14	*Retired (absorbed); not in text*	—
E16	Cross-family synthesis replication (Llama, Gemma)	Tab. 9
E18	Filter-vs-repair decomposition of the verification gate	Tab. 10
E19	Scale-ladder vs. a stronger learned baseline, OOD collapse	Tab. 11
E20	The complexity cliff: synthesis degrades past eight rules	Fig. 15
E21	Stochastic dynamics via seed-threaded verified synthesis	§4.14
E22	Planning: model-predictive lookahead through the program	Tab. 5
E23	Oracle-free error detection by ensemble disagreement	§4.14
E28	*Cut (setup only)*: atomic SWE-style repair suite	§ Setup
E29	*Cut (setup only)*: staged latent-defect repair suite	§ Setup
E37	Equal-information induction: code beats learning OOD	Tab. 4
E38	The induction ceiling is identifiability, not capability	Fig. 13
E43	Active induction: acting closes the identifiability gap	Fig. 14
E53	*Retired (sheaf section cut); not in text*	—
E61	Verified code vs a trained world model on control (absorbed into E63)	Tab. 6
E63	World-model bake-off: 6 runnable models × 2 domains	Tab. 6
E65	Cross-species on MiniGrid: verified code vs. DreamerV3 / V-JEPA-2	§4.13
E67	Continuous control: cartpole-swingup vs. learned/perceptual models	§4.13
E74	World-time compute: held-out lift vs. model size (diagnosis family)	§4.3
E76	World-count scaling of the world-time-compute lever	§4.4
E77	Coding-world transfer of world-time compute (pass@5)	§4.4
E78b	Label-exactness ablation isolates the causal driver	§4.4
E80	Real-data world-time compute on ARC-AGI (per-world TTT + ladder)	§4.5
E80	Replication across List Functions, CLRS, Bongard-RWR (vision boundary)	§4.6
E80	A descriptive regularity: LODO fit, tabular out-of-sample, learner-invariance	§4.8
E81	Frame-invariance across language frames (clean negative)	§4.9
E82	The hybrid loop: verified world as generator + inference backstop	§4.10
E83	Real-data cross-world transfer on CLRS-Text (weak null)	§4.8
E84	Cross-world transfer on a real domain (List Functions); generalize to held-out worlds	§4.7
E85	Trajectory vs. answer training unit on iterated worlds	§5

B Detailed result tables

Generator (family)	Runs	Verified acceptance (95% CI)	Probe acc. of accepted	Mean synthesis time
qwen2.5:7b (Qwen)	15	100% [0.80, 1.00]	0.98	—
qwen2.5:3b (Qwen)	15	100% [0.80, 1.00]	0.87	—
llama3.1:8b (Llama)	9	100% [0.70, 1.00]	0.85	9s
gemma2:9b (Gemma)	9	100% [0.70, 1.00]	1.00	4s

Table 9: E2 + E16: synthesis reliability by generator scale and family. Qwen rows: five compilation attempts per world; Llama/Gemma rows: three per world. Maximum four verify–repair iterations each; wall-clock includes all iterations.

Regime	Acceptance	Probe acc. (accepted)	Probe acc. (all runs)
Filter only (max_iters=1)	62%	0.62	0.39
Filter + repair loop (max_iters=4)	100%	0.65	0.65

Table 10: E18: filter-vs-repair decomposition of the full verification gate (3B generator, eight paired high-temperature attempts per regime).

Engine	1×	10×	100×
Synthesized code (0 transitions)	10/10	10/10	10/10
Delta-MLP, 128h (10k transitions)	5/10	0/10	0/10
MLP, 64h (10k transitions)	5/10	0/10	0/10
1-NN memorizer (10k transitions)	3/10	0/10	0/10
LLM next-state (7B, 0 transitions)	4/10	4/10	5/10

Table 11: E19: exact transition accuracy on the branch-covering probe suite across a 1×/10×/100× scale ladder.

Strategy	Seeds solving	Mean trials to first solve	Mean best score
random	10/10	4	17.60
tpe	10/10	5	17.60
random+refine	10/10	4	17.60

Table 12: E9: tuning-strategy comparison on the triage auto-configuration problem; ten seeds per strategy, 200-trial budget.

C The synthesized dynamics artifact

A representative accepted sprint-world transition, synthesized by `qwen2.5:7b` and verified by the full gate—the artifact whose exactness underwrites Table 1:

Accepted `transition()` (verbatim)

```
def transition(state, action):
    next_state = dict(state)
    name = action["name"]
    if name == "ship" and next_state["backlog"] > 0:
        next_state["backlog"] -= 1
        next_state["shipped"] += 1
        next_state["debt"] += 1
        next_state["bugs"] += next_state["debt"] // 4
    elif name == "fix":
        next_state["bugs"] = max(0, next_state["bugs"] - 2)
    elif name == "refactor":
        next_state["debt"] = max(0, next_state["debt"] - 2)
    return next_state
```

D Example tasks and failure cases

The coding-world tasks pair a buggy function with hidden tests; e.g. `dedupe` returns `list(set(xs))` (order-destroying) and must preserve first-seen order, and `balanced` counts parenthesis depth but misses early-imbalance strings like `")("`. Failure modes observed in the campaign: the 1.5B baseline resubmitted near-identical patches until the attempt budget expired on tasks where its first reading

of the spec was wrong—precisely the local minimum that high-temperature candidate sampling plus judge selection escapes; conversely, when several candidates passed, the judge's pick was strongly order-sensitive (E13: raw choices matched under order reversal on only 28% of rounds) while its accuracy was not—evidence that judge criteria and presentation order need auditing even when outcomes look fine. In E7/E15, the rubric judge compressed mid-range episodes toward similar scores (rank correlation 0.75 rather than 1.0, dropping to 0.58 under paraphrase), consistent with known LLM-judge coarseness [51].

E Reproducibility

One pipeline rebuilds every artifact: experiment scripts in `experiments/` write `results/*.json` (seeds fixed in-source); `python3 scripts/make_paper_assets.py` regenerates every figure, table, and the `numbers.tex` macros from those JSON files; and `make` in `paper/` builds this PDF. A continuous-integration workflow runs the offline test suite, re-runs the asset pipeline, and fails if `numbers.tex` would change, so the paper's numbers cannot silently drift from the committed results. `experiments/MANIFEST.md` classifies every experiment as deterministic-offline or Ollama-dependent and records its model and provenance; `experiments/requirements.lock` pins the analysis stack (Python, numpy, matplotlib). The 9 Ollama model snapshots that appear across the runs are `gemma2:9b`, `gpt-oss:20b`, `llama3.1:8b`, `qwen2.5:1.5b`, `qwen2.5:3b`, `qwen2.5:7b`, `qwen2.5:14b`, `qwen2.5-coder:7b`, `qwen3-coder:30b` (quantization `Q4_K_M`); platform, versions, and timestamp are recorded in each result file. Deterministic-offline experiments reproduce bit-for-bit; the Ollama-dependent runs reproduce in distribution (Metal is not bit-deterministic even at fixed seed), with dataset/model-pinned frozen artifacts where exact numbers are cited.

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.

[2] Ekin Akyürek, Mehul Damani, Linlu Qiu, Han Guo, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for abstract reasoning. *arXiv preprint arXiv:2411.07279*, 2024.

[3] Eloi Alonso, Adam Jelley, Vincent Micheli, et al. Diffusion for world modeling: Visual details matter in Atari. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[4] Thomas Anthony, Zheng Tian, and David Barber. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[5] Mido Assran et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. arXiv preprint arXiv:2506.09985, 2025. Joint-embedding predictive world model; predicts in representation space, not pixels.

[6] Jake Bruce, Michael Dennis, Ashley Edwards, et al. Genie: Generative interactive environments. *arXiv preprint arXiv:2402.15391*, 2024.

[7] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[8] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[9] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC prize 2024: Technical report. arXiv preprint arXiv:2412.04604, 2024.

[10] François Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems. arXiv preprint arXiv:2505.11831, 2025.

[11] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020.

[12] Jade Copet et al. CWM: An open-weights LLM for research on code generation with world models. *arXiv preprint arXiv:2510.02387*, 2025. 32B open-weights neural code world model; mid-trained on Python execution and Docker container traces, then multi-task RL.

[13] Nicola Dainese, Matteo Merler, Minttu Alakuijala, and Pekka Marttinen. Generating code world models with large language models guided by Monte Carlo tree search. *arXiv preprint arXiv:2405.15383*, 2024.

[14] Maxence Faldor, Jenny Zhang, Antoine Cully, and Jeff Clune. OMNI-EPIC: Open-endedness via models of human notions of interestingness with environments programmed in code. *arXiv preprint arXiv:2405.15568*, 2024.

[15] Fei-Fei Li and World Labs. Large world models and spatial intelligence. World Labs, 2025.

[16] Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, et al. Reinforced self-training (ReST) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.

[17] Daya Guo, Dejian Yang, Haowei Zhang, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[18] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

[19] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[20] Bingyi Kang, Yang Yue, Rui Lu, et al. How far is video generation from world model: A physical law perspective. *arXiv preprint arXiv:2411.02385*, 2024.

[21] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tülu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

[22] Wolfgang Lehrach et al. Code world models for general game playing. *arXiv preprint arXiv:2510.04542*, 2025.

[23] Kenneth Li, Fernanda Viégas, Martin Wattenberg, et al. What does it mean for a neural network to learn a "world model"? *arXiv preprint arXiv:2507.21513*, 2025.

[24] Wen-Ding Li, Keya Hu, Carter Larsen, Yuqing Wu, Simon Alford, Caleb Woo, Spencer M. Dunn, Hao Tang, Wei-Long Zheng, Yewen Pu, and Kevin Ellis. Combining induction and transduction for abstract reasoning. *arXiv preprint arXiv:2411.02272*, 2024.

[25] Mikołaj Małkiński, Szymon Pawlonka, and Jacek Mańdziuk. Reasoning limitations of multimodal large language models. a case study of Bongard problems. *arXiv preprint arXiv:2411.01173*, 2024. Introduces Bongard-RWR; ICML 2025.

[26] Larisa Markeeva, Sean McLeish, Borja Ibarz, Wilfried Bounsi, Olga Kozlova, Alex Vitvitskyi, Charles Blundell, Tom Goldstein, Avi Schwarzschild, and Petar Veličković. The CLRS-Text algorithmic reasoning language benchmark. *arXiv preprint arXiv:2406.04229*, 2024.

[27] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations (ICLR)*, 2023.

[28] Vincent Micheli, Eloi Alonso, and François Fleuret. Efficient world models with context-aware tokenization. *arXiv preprint arXiv:2406.19320*, 2024.

[29] Open-Ended Learning Team, Adam Stooke, Anuj Mahajan, et al. Open-ended learning leads to generally capable agents. *arXiv preprint arXiv:2107.12808*, 2021.

[30] OpenAI. Learning to reason with LLMs. OpenAI technical report, 2024.

[31] OpenAI. Video generation models as world simulators. OpenAI technical report, 2024.

[32] Szymon Pawlonka, Mikołaj Małkiński, and Jacek Mańdziuk. Bongard-RWR+: Real-world representations of fine-grained concepts in Bongard problems. *arXiv preprint arXiv:2508.12026*, 2025.

[33] Wasu Top Piriyakulkij, Yichao Liang, Hao Tang, et al. PoE-World: Compositional world modeling with products of programmatic experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[34] Julian Quevedo, Ansh Kumar Sharma, Yixiang Sun, et al. WorldGym: World model as an environment for policy evaluation. *arXiv preprint arXiv:2506.00613*, 2025.

[35] Santhosh Kumar Ravindran. Moral anchor system: A predictive framework for AI value alignment and drift prevention. *arXiv preprint arXiv:2510.04073*, 2025.

[36] Thomas M. Scanlon. *What We Owe to Each Other*. Harvard University Press, 1998.

[37] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.

[38] James Schwoebel, Ingrida Semenec, Jenia Rousseva, Marcos Ortiz, Collin Overbay, Manish Bhatt, Rome Thorstenson, and Martin G. Frasch. World Computing—Prototyping & Evaluating Verified Code World Models, 2026. arXiv preprint, in review.

[39] James Schwoebel, Ingrida Semenec, Jenia Rousseva, Marcos Ortiz, Collin Overbay, Christopher Klaus, Manish Bhatt, Rome Thorstenson, Jessica Tsai, and Martin G. Frasch. Solving ARC-AGI-3 with Perceptual Code World Models, 2026. arXiv preprint, under review.

[40] Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, et al. Beyond human data: Scaling self-training for problem-solving with language models. *Transactions on Machine Learning Research (TMLR)*, 2024.

[41] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

[42] Austin Spizzirri. The specification trap: Why static value alignment alone is insufficient for robust alignment. *arXiv preprint arXiv:2512.03048*, 2025.

[43] Aarohi Srivastava et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research (TMLR)*, 2023.

[44] Hao Tang, Darren Key, and Kevin Ellis. WorldCoder, a model-based LLM agent: Building world models by writing code and interacting with the environment. *arXiv preprint arXiv:2402.12275*, 2024.

[45] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.

[46] Keyon Vafa, Justin Y. Chen, Ashesh Rambachan, et al. Evaluating the world model implicit in a generative model. *arXiv preprint arXiv:2406.03689*, 2024.

[47] Petar Veličković, Adrià Puigdomènech Badia, David Budden, Razvan Pascanu, Andrea Banino, Misha Dashevskiy, Raia Hadsell, and Charles Blundell. The CLRS algorithmic reasoning benchmark. In *International Conference on Machine Learning (ICML)*, 2022.

[48] Zhouliang Yu, Yuhuan Yuan, Tim Z. Xiao, et al. Generating symbolic world models via test-time scaling of large language models. *arXiv preprint arXiv:2502.04728*, 2025.

[49] Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2023.

[50] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[51] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[52] Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, et al. Agent-as-a-judge: Evaluate agents with agents. *arXiv preprint arXiv:2410.10934*, 2024.

[53] Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, Xuekai Zhu, Haozhan Li, Yuchen Zhang, Xinwei Long, Ermo Hua, Biqing Qi, Youbang Sun, Zhiyuan Ma, Lifan Yuan, Ning Ding, and Bowen Zhou. TTRL: Test-time reinforcement learning. *arXiv preprint arXiv:2504.16084*, 2025.